

CPT102 Data Structures and Algorithms

数据结构和算法

内容基于 LearningMall 的 PPT。仅供学习和交流使用。任何机构或个人不得将其用于商业用途或未经授权的传播。如需引用或分享，请注明出处并保留此声明。



GITHUB

REPOSITORY



VISIT NOTES AUTHOR'S PAGE

CLICK ME

■ [Week 1 Lecture 0]

课程情况

Steven Guan	Room SD425	steven.guan@xjtlu.edu.cn
Kok Hoe WONG	Room SD431	kh.wong@xjtlu.edu.cn

课程占比

1. 5学分
2. 共2次assessments, 占比共20%
3. 一个 2 小时的考试, 占比 80%

Q&A

1. Name three typical type of data values in common data collections.

三种集合的典型数据类型值：整数（Integer）、字符串（String）、布尔值（Boolean）

2. Name three typical structures seen in common data collections.

三种典型的结构：数组（Array）、链表（Linked List）、栈（Stack）

3. Name three typical operations seen in common data collections.

三种典型操作：插入（Insert）、删除（Delete）、搜索（Search）。

4. Why do we learn data structure?

为什么我们学习数据结构？数据结构帮助我们有效地存储（storage）和管理（manage）数据，优化（optimize）数据处理的性能，是计算机科学的基础。

5. Why do we insist an algorithm must terminate?

为什么我们要求算法必须终止？

- 算法的主要目的是解决问题，而问题需要有一个明确的解决方案。
 - 如果算法不终止，它将无限运行，无法提供结果，从而无法达到解决问题的目的。
 - 终止性 (Termination) 是算法有效性 (Effectiveness) 和实用性 (Practicality) 的基本要求。

6. Why do we insist an algorithm must be precise?

为什么我们要求算法必须精确？精确的指令确保算法能够被正确执行，避免歧义 (ambiguity) 或错误。

7. Why instructions in an algorithm are written in a sequence?

为什么算法中的指令是按顺序编写的？顺序性 (Sequentially) 确保算法步骤的逻辑清晰，便于理解和执行。

■ [Week 1 Lecture 1]

数据结构是一种 **系统化的**(Systematic) 方式，用于组织数据集合以便**高效访问**(Efficiency access)。

每种数据结构都需要多种算法来处理其中的数据，因此，我们从以下方面研究数据结构

1. **属性** (properties)
2. **组织方式** (organisation)
3. **操作** (operations)

为什么我们需要数据结构？

虽然 原始类型 (例如 `int`, `double` 这种) 和 简单数组 可以满足基本需求，但数据结构提供了**更高级、更结构化的方式**来表达和处理复杂信息，使算法描述更清晰高效

我们将会探讨如下数据结构

1. 数组 **Arrays**
2. 列表 **Lists**
3. 栈 **Stacks**
4. 队列 **Queues**
5. 映射 **Maps**
6. 树 **Trees**
7. 图 **Graphs**

好的数据结构能优化效率，做出更好的软件。

因此，接下来我们探讨，有助于开发高质量软件的主要原则：**抽象** (abstraction)、**信息隐藏** (information hiding)、**封装** (encapsulation)。以及**静态结构**、**动态结构** (static, dynamic structures) 之间的选择。

高质量软件3大原则

一个抽象的例子是，当我们查看导航时，地图绘制的是一片森林，而不是一颗一颗的树木。

[1] 抽象

我们可以将 抽象 视为一个过程（process）或一个实体（entity）。

- 作为过程，抽象指的是提取（extracting）一个项目或一组项目的基本细节，同时忽略非必要的细节。
- 作为实体，抽象指的是一个模型、视图或某种实际项目的表示。

在软件开发的背景下，我们又可以把 抽象 视为：

- 数据抽象（data abstraction）：确定数据存储和操作方式中哪些细节是重要的，哪些是不重要的。
- 过程抽象（procedural abstraction）：确定任务完成方式中哪些细节是重要的，哪些是不重要的。

抽象的关键：提取组件（components）的共性（commonality）并隐藏其细节。抽象通常关注对象/概念的外部视图（outside view）。

一个抽象的例子就是：**哈夫曼编码**（Huffman coding）。它是一种有效的数据编码方法，常用于通信。编码存储在**码本**（Code Book）中。在所有情况下，必须传输 码本 和 编码数据（Encoded data） 以进行解码。

哈夫曼编码

要了解霍夫曼编码是如何工作的，建议看 [这个视频（这个视频讲的非常好!）](#)。当然，接下来是用图文的形式简单介绍哈夫曼编码。

二进制编码

假设我们需要编码串 A B A A C D C，那么我们可以用二进制编码。在这里，字母出现的顺序分别是 A B C D，因此他们可以用二进制编码：

字母	二进制
A	0
B	1
C	10
D	11

然后这个 A B A A C D C 就可以被加密为 0 1 0 0 10 11 10 也就是 0100101110。但是在解密的时候就会发生歧义。例如：组合 BA (10)可能被翻译为 C。这是因为 B 充当了 C 的前缀。

因此，为了解决歧义，引入了等长编码。

等长编码

字母	等长编码
A	00
B	01
C	10

字母	等长编码
D	11

这种编码的好处是一定不会出现前缀，每个字母对应的编码唯一，不会出现歧义。00 01 00 00 10 11 10

但是这种编码方式还是太长了。有什么编码方案能满足：

- 1. 编码出来最短
- 2. 没有歧义（能加密和唯一解密）

答案是：**哈夫曼编码**。

哈夫曼编码

哈夫曼编码的流程是：

- 1. 计算每个字母出现的次数。把这些次数当成**节点**
- 2. 每次相加两个最小的节点，构建成一个二叉树，其父节点是两个最小节点之和。
- 3. 继续步骤2，但是父节点也要算进去

这么看有点抽象，让我们来写一个例子。

还是这题，例如字符串 A B A A C D C 如何才能构建出哈夫曼编码？

步骤1：计算每个字母出现的次数

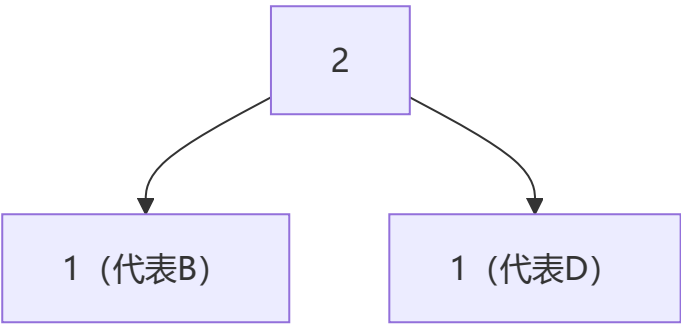
- A：3次
- B：1次
- C：2次
- D：1次

步骤2：构建哈夫曼树

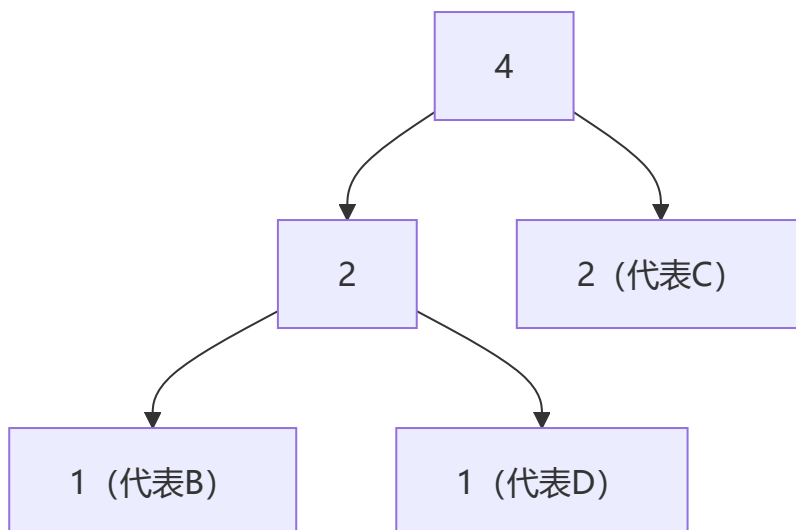
首先有四个节点，3，2，1，1，因此最小的两个节点是 1 和 1。



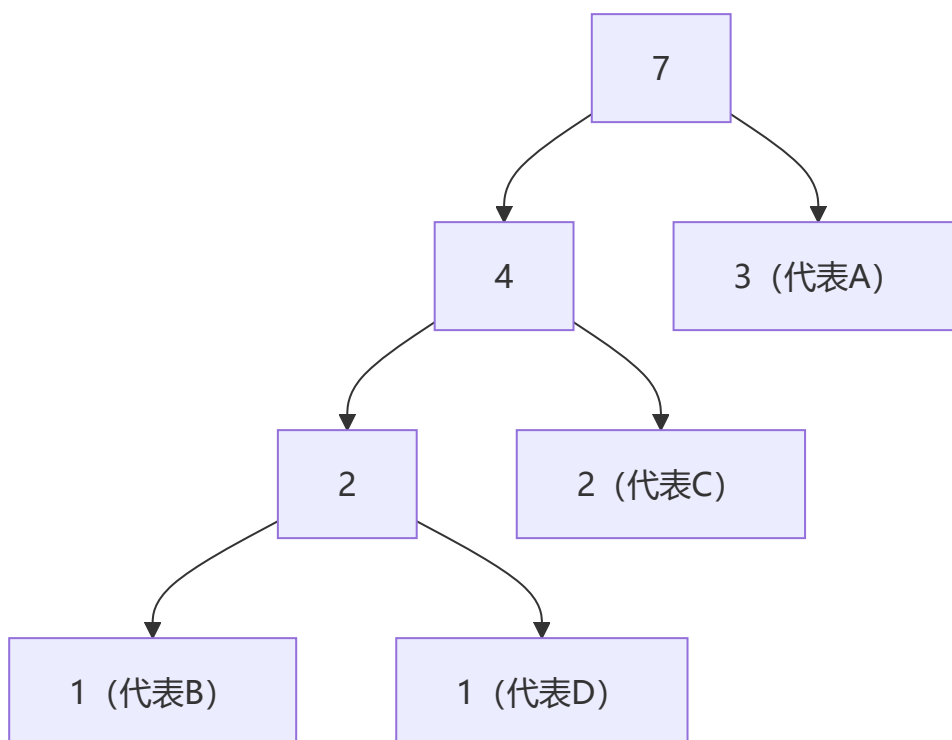
然后相加求出它们的父节点



接下来我们有还剩下三个节点：3（代表A）、2（代表C）、2（父节点）。因此最小的两个是2（代表C）和2（父节点）。把他们拼接。



最后，我们剩下两个节点：3（代表A）和4（父节点）。因此，把他们拼接。



最后，我们定义左子树的边为0，右子树的边为1。这样，通过二叉树遍历的方式就能得出，A,B,C,D的编码是：

- A：1
- B：000
- C：01
- D：001

因此，A B A A C D C 对应的哈夫曼编码是：1000110100101；

为什么哈夫曼编码能够解决前缀冲突（歧义）问题？因为所有的字母都在叶子节点上。假设A是B的前缀，那么A必须要在B的必经之路上。然而哈夫曼树的每一个字母都必然在叶子上，因此A不可能在B的必经之路上。这就解决了冲突问题。

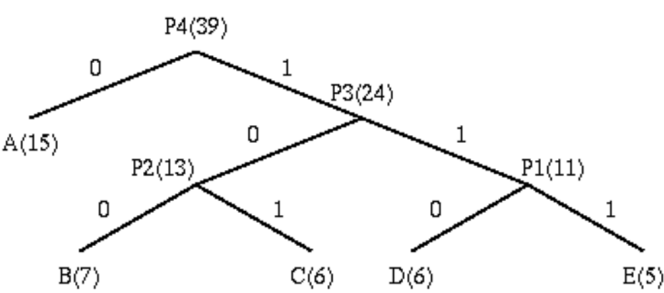
为什么哈夫曼编码能够最短？因为频率越大的字母越往树头靠，这样编码出来的就越短，压缩率更大。

值得注意的是，最开始我们操作 B 和 D 的时候，左右是没有讲究的。虽然左右会影响哈夫曼编码出来的值，但是并不影响其性质。也就是说，**对于一个串，哈夫曼编码的值不是唯一的。**

Subtotal (# of bits) 指的是**每个符号在编码后所占用的总位数**。实际上计算公式如下：

$$subtotal = frequency * length$$

这里的 frequency 就是频率，length 就是路径长度（就是走了多少步）。例如：



Symbol	Count	Code	Subtotal (# of bits)
A	15	0	15
B	7	100	21
C	6	101	18
D	6	110	18
E	5	111	15
TOTAL (# of bits):			87

这里的 A 对应的解码是 0，那么其 subtotal = 15 * 1 = 15。

B 的 subtotal = 7 * 3 = 21；如此往复。

哈夫曼编码使用了**优先队列**(Priority queues)和**二叉树**(Binary trees)。

[2] 信息隐藏

信息隐藏：指模块的使用者只需了解该模块的必要细节（通过**抽象**识别）

users of a module need to know only the essential details of this module

而抽象帮助我们识别出哪些细节重要，哪些不重要。

信息隐藏旨在将内部数据或信息屏蔽，避免被直接操控（direct manipulation）。

信息隐藏示例：汽车与驾驶员

汽车通过为驾驶员提供一个**标准化接口**（如踏板、方向盘、变速杆、信号灯、开关等），展示了信息隐藏的概念。这个接口隐藏了汽车内部的复杂机制（如发动机、变速箱、电子系统等）。驾驶员通过培训和考试学习使用这个接口，使他们能够操作任何汽车，而无需了解其内部工作原理。

或者Java中，通过**封装**的方法来进行信息隐藏。

[3] 封装

与抽象类似，我们可以将**封装**视为一种**过程**或一种**实体**

- 作为一种过程，封装意味着：将一项或多项（数据/函数）纳入一个**容器**中的行为。
- 作为一种实体，封装意味着：一个**容器**，其中包含一项或多项（数据/函数）。（该容器内部与外部的分隔有时被称为“**墙**(Wall)”或“**屏障**(Barrier)”）

面向对象（O-O, Object-Orient）中的封装

在面向对象编程中，封装是指将对象正常运行所需的全部资源（即方法和数据）包含在对象**内部**。其核心思想是：不要告诉我你是怎么做的；只要让我知道你能做什么！

通信（Communication）中的封装

在通信领域，封装是指将一个数据结构包含在另一个结构内，从而使第一个数据结构被隐藏。

例如，一个 TCP/IP 格式的数据包可以被封装在一个 ATM 帧中。封装的核心目的是将复杂的细节隐藏在一个更简单或标准化的形式中，以便于传输和处理。

[4] 总结

抽象 (Abstraction)、**信息隐藏 (Information Hiding)** 和 **封装 (Encapsulation)** 是不同的概念，但它们彼此相关。

- **抽象** 是一种帮助我们识别某个模块中哪些信息对用户是重要的、哪些是不重要的技术。
- **信息隐藏** 是一项原则（Principle），即所有不重要的信息都应该对用户隐藏。
- **封装** 则是将信息打包的技术，目的是隐藏那些应该隐藏的内容，同时暴露那些需要可见的内容。

抽象和**封装**在**信息隐藏**的指导原则下，我们享有以下优势：

1. 更简单、模块化的程序，更易于设计和理解
2. 消除或最小化**直接操作**(direct manipulation)数据带来的副作用
3. 让错误局部化（只有类上定义的方法才能操作类的数据），从而支持局部化测试
4. 程序模块更易于阅读、修改和维护（maintain）

[5] 静态和动态数据结构的选择

除了**时间**和**空间**效率之外，选择数据结构的另一个重要标准是：能存储的数据项数量是否可以根据我们的需求进行调整，或者是固定的。

动态数据结构：可以在运行时根据当前需求进行增长或缩小。

例如用于模拟交通流量的结构。

静态数据结构：在创建时就是固定的

例如，用于存储邮政编码或信用卡号码的结构（这些数据具有固定格式）

注意，静态（或动态）的是**结构**，而不是数据。

因此，在静态数据结构中，存储的数据可以随时间变化，但结构是固定的。

例如C++中的数组。一般而言，数组长度不能改变。

示例

静态数据结构的一个示例是**数组**：

```
1 | int[] a = new int[50];
```

这行代码分配了可以容纳50个整数的内存空间。

Java提供了 `new` 一词来向**编译器/解释器**指示需要分配多少空间以及哪种数据类型的空间。

即使我们编写以下代码，数组也总是动态分配的（Dynamically allocated）：

```
1 | int[] a = {10, 20, 30, 40};
```

然而，数组的大小是固定的。但问题是，什么叫**动态分配**？

在Java中，数组即使在编译时定义大小（例如 `int[] a = {10, 20, 30, 40};`），它们仍然是在**运行时分配内存**的。这意味着数组的内存空间是在程序**运行期间**通过堆内存动态获取的（`new`出来的都在堆内存中），而不是在编译时静态地确定和分配。这与静态内存分配的语言（如C中的静态数组）不同，后者在编译时已经确定了内存分配。

问题：Java数组是静态还是动态数据结构？

Java数组从**内存分配（Memory allocated）**的角度来看是**动态的**，因为它们的内存是在运行时分配的。

然而，数组的大小在初始化时（at the time of initialize）是固定的，因此在大小上表现出静态特性。

[6] 静态数据结构优点和缺点

优点

- 易于指定（Ease of specification）。编程语言通常提供一种简便的方式来创建**任意大小**的静态数据结构。
没有内存分配开销。

没有内存分配开销指：静态数据结构在运行时**不需要动态分配或释放内存**，从而避免了与动态内存管理相关的性能开销。

缺点：

由于静态数据结构的大小是固定的：

- 不存在可以用来扩展静态结构的操作；

2. 如果要对静态数据结构进行扩展（例如增加元素），就需要为其分配额外的内存空间，而分配内存的操作是需要时间的，因此会产生时间和性能上的消耗
3. 一方面，必须确保有足够的容量；另一方面，如果存储的数据项数量较少，那么静态数据结构的一部分将保持为空，但这些内存已经被分配，无法用于存储其他数据

[7] 动态数据结构的优点和缺点

优点

1. 无需事先知道确切的数据项数量
2. 内存空间的高效利用
 1. 在需要添加数据项时（如果这些数据项无法存储在当前结构中），可以扩展动态数据结构的大小。
 2. 在动态数据结构中存在未使用的空间时，可以收缩其大小。这样，结构总是能够保持恰到好处的大小，不会浪费任何内存空间。

缺点

1. 内存分配/释放开销：这意味着在使用动态数据结构（如链表、堆栈或队列）时，系统需要分配新的内存空间来存储增加的数据，或者释放不再需要的内存空间。这些操作会产生一定的性能开销。
2. 每当动态数据结构增长或缩小时，分配给数据结构的内存空间必须增加或减少（这需要时间）：这意味着随着动态数据结构的扩展或缩小，系统需要调整内存分配，这过程涉及到重新分配内存并可能移动数据，因此会花费额外的时间。

Q&A

1. 空间效率(Space efficiency)和时间效率(Time efficiency)都是用来评估算法（以及数据结构）性能的指标。（对还是错？）

这句话是正确的。空间效率衡量的是算法运行所需的最大内存空间，而时间效率衡量的是算法完成任务所需的时间。这两者都是评估算法性能的重要标准。

2. 动态数据结构通常更具空间效率。（对还是错？）

这句话是错误的。动态数据结构在内存使用上并不总是比静态数据结构更高效。相反，因为动态数据结构需要额外的内存来存储指针或链接，而且可能会导致内存碎片，所以静态数据结构在某些情况下可能更节省内存。

3. 静态数据结构通常更具有时间效率。（对还是错？）

这句话是正确的。静态数据结构（如数组）由于内存分配是预先确定的，不需要频繁的内存分配和释放操作，因此在访问和操作数据时通常更快，因此具有更高的时间效率。

4. 信息隐藏是指：软件组件的用户只需要知道如何初始化和访问组件的必要细节，而不需要了解实现的细节。（对还是错？）

对的。比如你开车，只需要知道如何初始化（启动发动机）和如何把握方向盘这些即可。不需要知道内燃机是如何工作的，这属于细节方面的。

总结

数据结构：

数据结构是计算机中**组织和存储数据的方式**，决定了数据的访问和处理效率。

抽象：

抽象是指隐藏复杂的实现细节，只关注问题的核心概念，是设计和理解数据结构的重要方法。

抽象的应用：

1. 数据挖掘 (Data Mining)
2. 大数据分析 (Big Data Analytics)
3. 数据库知识发现 (Knowledge Discovery in Databases / KDD)

哈夫曼编码与动态队列：

哈夫曼编码是一种用于**数据压缩的技术**，动态队列 (Dynamic queues) 是一种根据数据量动态调整大小的队列结构。

信息隐藏：

信息隐藏是指将实现**细节**与用户接口**分离**，用户只需知道如何使用组件，而不必了解其实现。

封装：

封装是将数据和操作数据的方法绑定在一起，**隐藏内部细节**，确保数据的安全性和一致性。

空间与时间效率：

空间效率是指算法所需的内存资源，时间效率是指算法执行所需的时间，两者都是评估算法性能的关键指标。

静态与动态数据结构：

静态数据结构（如数组）的大小在编译时确定，而动态数据结构（如链表）的大小可以根据需要动态调整，但动态数据结构通常会牺牲一些空间或时间效率。

■ [Week 2 Lecture 2]

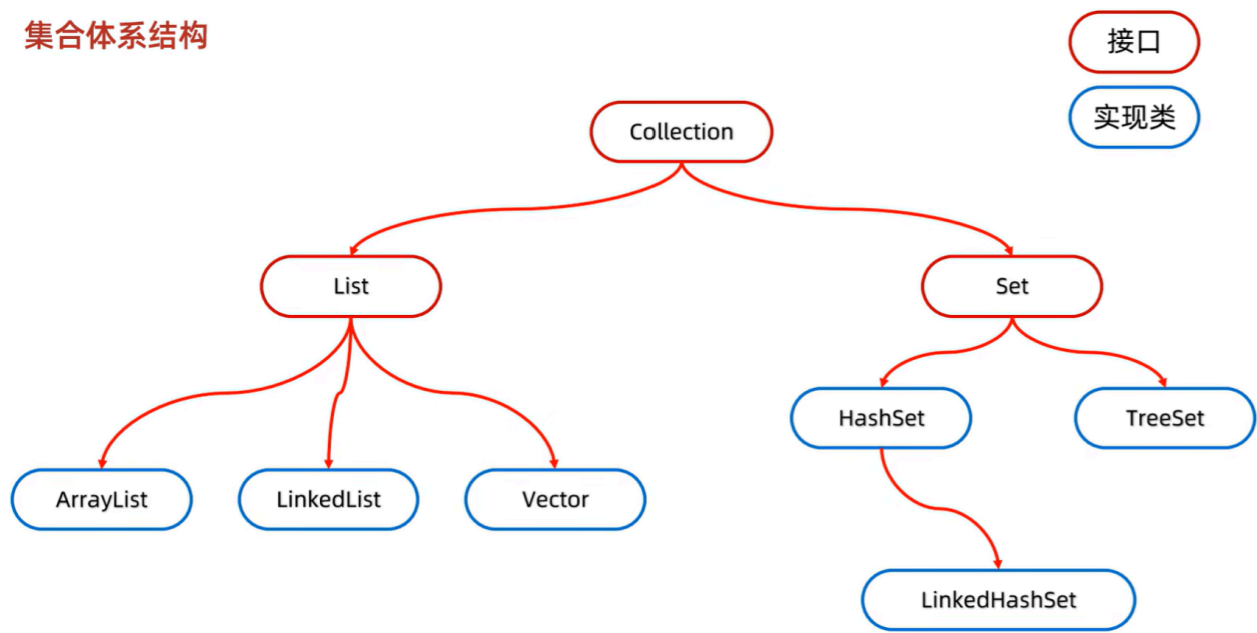
标题：使用 Java `Collection` 库

Lecture 2主要介绍了 Java 的一些功能。PPT非常乱，所以Lecture 2 进行了一些调整，至少看起来有些逻辑

1. 两种集合类型
2. 学会使用 Java 库进行编程
3. 集合
4. 使用**对象列表 (Lists of Objects)** 进行编程

[0] 非常重要的一张图

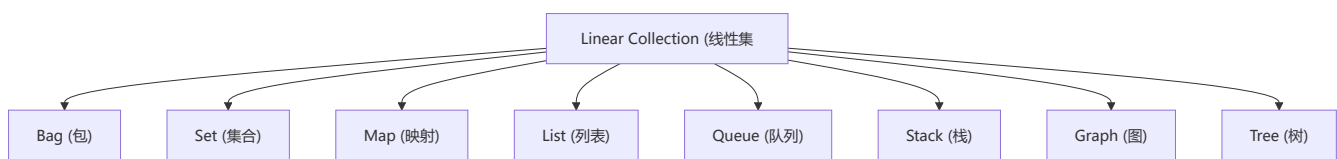
集合体系结构



这张图PPT上没有。而且PPT讲的东一块西一块，如果你能时刻看懂这张图，可能会对你的学习有所帮助。

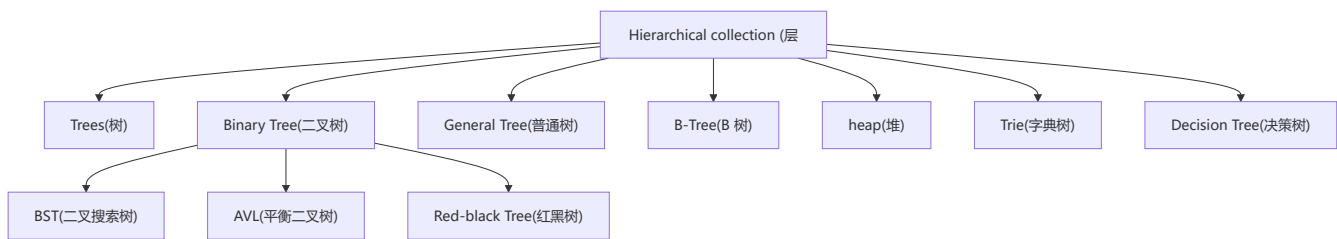
[1] 两种集合类型

1. 线性集合(Linear Collection)的种类有很多：



使用线性集合来编程，实现数据的搜索与排序，实现链表数据结构与“指针”

2. 层次化集合 (Hierarchical collections) 的种类也有很多：



PPT中只列出Tree、Binary Tree、General Tree。但其余的数据结构也很重要。例如决策树用于机器学习。能利用AVL树加泛型 `<E>` 在Java中创建集合。

使用层次化结构，实现树的构建、搜索、遍历 (Traversing) ...

[2] 学会使用库进行编程

因为例如GUI、网络程序，规模过大无法从头搭建，因此学会使用库进行编程尤为重要

Java 提供了很多标准工具类。

java.util 集合类

java.io 输入输出类

javax.swing java.awt 用于 GUI 程序的大型类库...

泛型

泛型 (Generics)

接下来用一个例子来介绍泛型 `<E>`:

```
1 public class MyLists<E>{           // 定义一个泛型类
2     private ArrayList<E> list = new ArrayList<>();
3
4     public E getObject(int index){ // 返回第 index 个元素
5         return list.get(index);
6     }
7     public void add(E element) // 添加元素
8     {
9         list.add(element);
10    }
11 }
```

`MyLists` 中什么都能存, 所以 `getObject` 方法返回的类型不确定, 所以就填 `<E>`。但是一旦 `MyLists` 的对象被创建, 虚拟机就知道这个泛型到底代表着什么。例如 `MyLists<Integer> list = new MyLists();` 虚拟机就知道这个泛型 `E` 代表 `Integer`。如果在编译前试图插入非 `Integer` 数据, IDE就下标会红线报错。

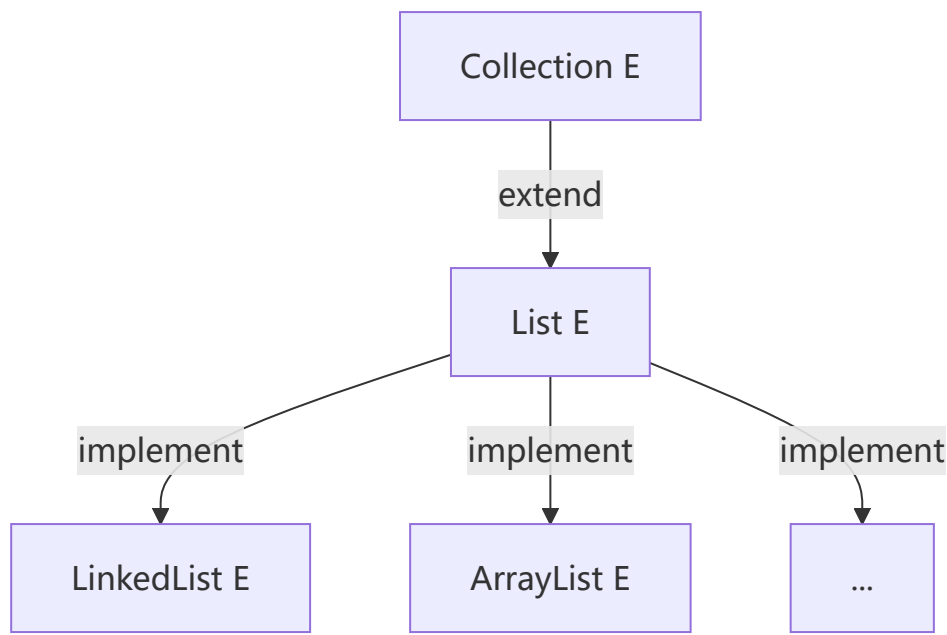
当然如果不写, 那就是默认为 `Object`。这时候集合里面什么都能放。不会有任何限制。

因此这就是泛型的作用。接下来是对应的例子:

```
1 public static void main(String[] args) {
2     // 没有指定数据类型, 默认Object什么都能放
3     MyLists listNoSpecific = new MyLists();
4
5     listNoSpecific.add(1); // 添加 Integer
6     listNoSpecific.add(new Student("zhangsan", 13)); // 添加自定义引用数据类型
7     System.out.println(listNoSpecific.getObject(0));
8     System.out.println(listNoSpecific.getObject(1));
9     // 输出:
10    // 1
11    // Student{name='zhangsan', age=13}
12
13    // 如果确定了数据类型。那么泛型E就会记住, 你这个对象只能存Integer类型
14    MyLists<Integer> listSpecific = new MyLists<>();
15
16    listSpecific.add(1);           // 可插入, 不报错
17    listSpecific.add(new Student("zhangsan", 13)); // 报错!!
18 }
```

接口和实现类

Java 中存在接口继承接口。例如 `Collection <E>` 下就被 `List<E>` 继承。而 `List<E>` 接口能被实现为 `ArrayList<E>` 和 `LinkedList<E>`。



Java 中的 `Collection`（接口）等类非常抽象（Abstract），因为他实现类接受泛型 `<E>`（也就是什么数据类型都能存）。在这里，`<E>` 相当于一个记录**变量类型**的变量。

```
1 public interface Set<E> extends Collection<E> {
2     public void add(??? item); /*...描述...*/
3     public void remove(??? item); /*...描述...*/
4     public boolean contains(??? item); /*...描述...*/
5     ...
6     // （Java Set 接口中还有许多其他方法）
7 }
```

Java 接口中**没有构造函数（无法实例化）**，**没有字段（无法储存数据）**，**没有方法体（空的，等实现类 `@Override`）**。

作为接口，其对应的实现类对象有很多。

- 1. 不关心元素的类型是什么。
- 2. 不关心它们内部如何存储或操作。

接口	说明
<code>Collection</code>	包（最通用的集合类型）
<code>List</code>	有序集合

接口	说明
<code>Set</code>	无序集合，不允许重复元素
<code>Queue</code>	有序集合，访问受限（从一端添加，从另一端移除）
<code>Map</code>	键值对 key-value （或映射 Mapping）

实现类	类名
List 类	<code>ArrayList</code> , <code>LinkedList</code> , <code>Vector</code> , ...
Set 类	<code>HashSet</code> , <code>TreeSet</code> , ...
Map 类	<code>HashMap</code> , <code>TreeMap</code> , ...
其他	...

[3]Q&A

1. Name 3 type of collections that can be implemented under Java Programming with Linear collections

在 Java 编程中，可以被实现（这里的实现是实现接口 implement interface 的意思）的线性集合类型的三种类型是：

答案：列表（List）、队列（Queue）、栈（Stack）、...（参考 **[1] 两种集合类型** 中的 线性集合）

2. Name 3 operations that can be implemented under Java Programming with Linear collections

Java 中能被实现的线性集合的三种操作是：

答案：`add()` 添加元素，`remove()` 删除元素，`get()` 获取元素。

3. Name 2 type of collections that can be implemented under Java Programming with Hierarchical collections

在 Java 编程中，可以被实现的层次化集合类型的两种类型是：

答案：

1. Tree（基于树的集合）
2. TreeMap（基于树的映射）

4. Name 4 operations that can be implemented under Java Programming with Hierarchical collections

Java 中能被实现的层次化集合的四种操作是

答案：`put()` 添加键值对，`remove()` 删除键值对，`get()` 获取值，`traverse()` 遍历集合。

5. What is a software “library”?

什么是软件“库”？

答案：软件库是一组预先编写好的代码、函数或类，供开发者在编写程序时调用，以减少重复工作和提高开发效率。例如 Java 的各种包，比如 `java.lang` 是 Java 的标准库之一。

6. Define Java “Package”.

定义 Java 的包

答案: Java 包 (Package) 是用于组织类 (Class) 和接口 (Interface) 的命名空间, 类似于文件夹。它用于避免命名冲突, 并提供访问控制。

例如,

```
1 class Tools{
2     protected static String HelloWorld(){
3         System.out.println("HelloWorld")
4     }
5 }
```

Tools 中的 `HelloWorld()` 方法能被所有的继承子类 (`extend` 的子类) 和所有在一个包下的类访问。而其他包下的非子类则不能访问。

7. Name Java's IO library.

Java 的 `IO` 库是什么?

答案: Java 的 `IO` 库是 `java.io`, 用于处理输入输出操作, 如文件读写、流操作等。

8. Name Java's GUI library.

Java 的 `GUI` 库是什么?

答案: Java 的 `GUI` 库是 `java.awt` (抽象窗口工具包) 和 `javax.swing` (Swing 框架)。

9. What is the Java statement to include package or class into your program?

用于将包或类导入到程序中的 Java 语句是什么?

答案: 使用 `import` 语句, 例如:

```
1 import java.util.ArrayList; // 导入 ArrayList 类
2 import java.util.*;         // 导入 java.util 包中的所有类
```

■ [Week 2 Lecture 3]

Lecture 3 讲了 `Collection` 和 `List`, 以及迭代器。

上个 Lecture 中介绍了 `Collection<E>` 接口, 以及继承它的接口 `List<E>`。子接口必须拥有父接口的所有方法。同时可以适当增加属于自己的方法。

对于 `List` 这些线性集合, 实际上是有索引的。但是对于一些 `set` 集合, 他们必须使用其他方法进行遍历 (比如迭代器)。PPT上只说了用 `Iterator` (迭代器) 遍历, 实际上, 对于 `collection` 下的实现类, 一共有三种遍历方式。

三种遍历方式

1. 迭代器遍历
2. 增强 `for` 遍历
3. Lambda表达式遍历

迭代器遍历

迭代器不依赖索引，其类叫 `Iterator`。

迭代器遍历时，不能用集合的方法进行增加或者删除，会发生并发修改异常

`ConcurrentModificationException`。（非常重要，但是PPT没提）

如果需要删除，那么就用迭代器提供的 `.remove()` 方法进行删除。如果要添加元素，现在还没有办法。

Collection集合获取迭代器

方法名称	说明
<code>Iterator<E> iterator()</code>	返回迭代器对象，默认指向当前集合的0索引

Iterator中的常用方法

方法名称	说明
<code>boolean hasNext()</code>	判断当前位置是否有元素，有元素返回true，没有元素返回false
<code>E next()</code>	获取当前位置的元素，并将迭代器对象移向下一个位置。

- ★ `.hasNext()` 这个是判断当前位置是否有元素，而不是下一个是否有元素。
- ★ `.next()` 这个会做2个事情，一个是获取当前元素，然后移动指针到下一位。

```
1 Collection<String> coll = new ArrayList<>();
2 coll.add("123");
3 coll.add("456");
4 coll.add("789");
5 coll.add("000");
6 Iterator<String> it = coll.iterator(); // 新建迭代器
7 while (it.hasNext()){
8     System.out.println(it.next());
9 }
```

增强 for 遍历 (推荐)

- 增强 `for` 的底层就是迭代器，为了简化迭代器的代码书写的。
- 它是JDK5之后出现的，其内部原理就是一个 `Iterator` 迭代器
- 所有的单列集合和数组才能用增强for进行遍历。

格式：

```
1 for (元素的数据类型 变量名 : 数组或者集合) {
2 }
```

示例：


```
1 for (String s : list) {
2     System.out.println(s);
3 }
4 // list.for 按下tab自动补全
```

细节：修改增强 `for` 的第三方变量不会改变原本集合中的数据。

Lambda表达式遍历

得益于JDK 8开始的新技术Lambda表达式，提供了一种更简单、更直接的遍历集合的方式。

方法名称	说明
<code>default void forEach(Consumer<? super T> action)</code>	结合lambda遍历集合

这里的Consumer是接口，我们可以利用匿名内部类的形式。

```
1 coll.forEach(new Consumer<String>() // 接口
2     // 匿名内部类
3     {
4         @Override
5         public void accept(String s) { // 这里的s就是依次表示集合中的每个数据。
6             // 这里直接操作s即可
7         }
8     });
9
```

`forEach` 的底层原理，其实也会自己遍历集合然后得到每一个元素，然后交给accept方法。

简化后的匿名内部类，也就是Lambda表达式可以这样写：

```
1 coll.forEach(s -> System.out.println(s));
```

■ [Week 3 Lecture 4]

[1] 包 Bags

包 (Bags)

包是一种集合，具有以下特性：

1. 不维护结构或顺序
2. 无访问限制（可随时访问任何元素）
3. 允许重复元素

基本操作：

1. `add(value)` → 如果集合被修改则返回 `true`

2. `remove(value)` → 如果集合被修改则返回 `true`
3. `contains(value)` → 如果值在包中则返回 `true`
(使用 `equals` 进行测试)
4. `findElement(value)` → 如果值在包中则返回匹配的元素

附加操作:

```
size(), isEmpty(), iterator(), clear(),  
addAll(collection), removeAll(collection),  
containsAll(collection), ...
```

包的应用

何时使用包? 当集合不需要排序, 且可能包含重复元素时:

1. 当前登录用户的集合 (可能有重复吗?)
2. 书籍收藏中的书籍 (可能有重复吗?)
3. ...

包没有标准的实现!

[2] 集合 Set

集合 (Set ADT)

集合是一种具有以下特性的集合:

1. 不维护结构或顺序
2. 无访问限制 (可随时访问任何元素)
3. 唯一特性是排除重复元素

操作: (与包类似, 但行为不同)

1. `add(value)` → 如果值被添加则返回 `true` (即没有重复)
2. `remove(value)` → 如果值被移除则返回 `true` (值在集合中)
3. `contains(value)` → 如果值在集合中则返回 `true`
4. `findElement(value)` → 如果值在集合中则返回匹配的元素

`findElement(value)` → 如果值在集合中则返回匹配的元素

这里的“匹配的元素”指的是集合中与给定值 `value` 相等的元素。由于集合不允许重复元素, 如果 `value` 存在于集合中, `findElement(value)` 会返回集合中的那个元素 (即与 `value` 相等的元素)。如果 `value` 不在集合中, 则返回 `null` 或某种表示未找到的值。

例如:

- 假设集合 `S = {1, 2, 3, 4, 5}`, 调用 `findElement(3)` 会返回 `3`, 因为 `3` 在集合中。
- 调用 `findElement(6)` 会返回 `null`, 因为 `6` 不在集合中。

这个操作通常用于检索集合中具体的元素, 而不是仅仅检查元素是否存在 (`contains(value)` 用于检查存在性)。

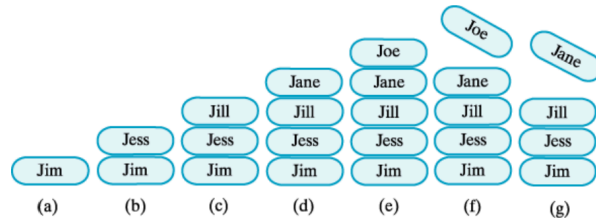
5. ...

集合与列表一样常见

[3] 栈 (Stack)

- 栈根据元素添加的顺序来组织条目
- 添加操作总是在一端进行，称为栈顶(Top)
- 最近添加的元素始终位于栈顶

(a) 执行 push 操作，添加 Jim;
(b) 执行 push 操作，添加 Jess;
(c) 执行 push 操作，添加 Jill;
(d) 执行 push 操作，添加 Jane;
(e) 执行 push 操作，添加 Joe;
(f) 执行 pop 操作，检索并移除 Joe;
(g) 执行 pop 操作，检索并移除 Jane;



栈 (Stacks) 是一种特殊的列表 (List)：

- 它是一个值的序列 (“序列”意味着元素是按顺序排列的)。
- 受限的访问方式：只能从一端进行添加、获取和移除操作。
- 存在栈接口及其不同的实现 (如 `ArrayStack`、`LinkedStack` 等)。

在 Java 集合库中：

- `Stack` 是实现了 `List` 接口的类。
- 它提供了额外的操作：`push(value)`、`pop()` 和 `peek()`：
 - `push(value)`：将值压入栈顶。
 - `pop()`：移除并返回栈顶的元素。
 - `peek()`：返回栈顶的元素，但不移除它。
- 此外，它还支持其他列表操作。

Stack 类

java.util 包中 Stack 类的方法

```
1 // 将元素 item 压入栈顶
2 public void push(Object item);
3
4 // 弹出并返回栈顶元素
5 public Object pop();
6
7 // 返回栈顶元素但不弹出
8 public Object peek();
9
10 // 判断栈是否为空
11 public boolean isEmpty();
12
```

```
13 // 清空栈
14 public void clear();
```

关于 StackInterface

```
1 public interface StackInterface
2 {
3     /** 任务：将新条目添加到栈的顶部。
4      * @param newEntry 要添加到栈中的对象 */
5     public void push(Object newEntry);
6
7     /** 任务：移除并返回栈顶部的对象。
8      * @return 栈顶部的对象，如果栈为空则返回 null */
9     public Object pop();
10
11     /** 任务：获取栈顶部的对象。
12      * @return 栈顶部的对象，如果栈为空则返回 null */
13     public Object peek();
14
15     /** 任务：判断栈是否为空。
16      * @return 如果栈为空则返回 true */
17     public boolean isEmpty();
18
19     /** 任务：移除栈中的所有条目 */
20     public void clear();
21 } // 结束 StackInterface
22 Specifications of the ADT Stack
23 • Specification of a stack of objects
```

[4] 栈的应用

- 处理结构化（嵌套）数据文件：
 - 例如，读取带有结构化标记的文件（如 HTML、XML 等）。
- 程序执行：
 - 例如，处理子任务后返回到先前的任务。
- 编辑器中的撤销操作：
 - 栈用于记录操作历史以支持撤销功能。
- 表达式求值：

- 例如，计算表达式 $(6 + 4) * \left((12.1 * \sin(15)) - \left(\frac{\cos(20)}{38} \right) \right)$ 。

Example: 如何确保网页文档中的 XML/HTML 标签正确嵌套？

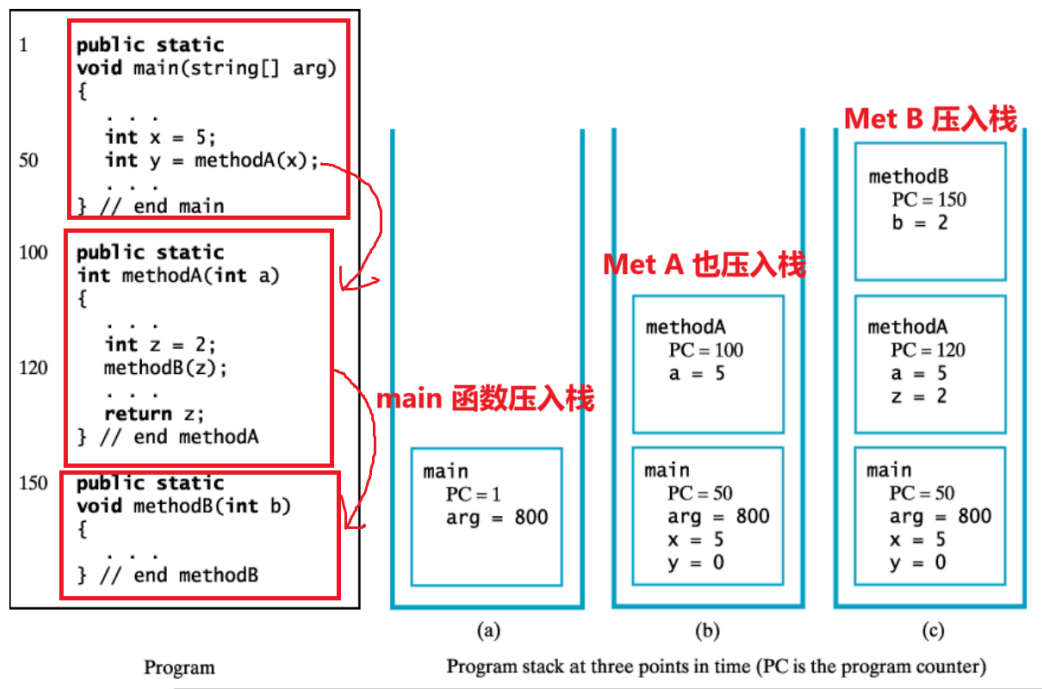
答案：使用栈！

程序执行中的程序栈

当一个方法被调用时

- 运行时环境 会创建一个**活动记录(activation record)**来展示方法在执行期间的状态
- 活动记录被压入程序栈 (Java 栈)
- 栈顶属于**当前正在执行的方法**，栈中的下一个记录属于调用当前方法的方法

这里的**方法**指的是Java中的函数，或者叫做method，就例如 `public static void calculate(int a, int b){return a+b}` 这种



如果无人调用Met B那么就弹出栈，再弹出A，最后弹出main

递归方法

一个递归方法进行多次递归调用时:

- 会在程序栈中放置多个活动记录
- 这解释了为什么递归方法可能会占用大量内存
- 可以通过使用栈将**递归recursion**替换为**迭代iteration**

用于表达式求值

例如表达式: $(6 + 4) * ((12.1 * \sin(15)) - (\frac{\cos(20)}{38}))$

它是如何工作的?

使用栈处理代数表达式

- 检查中缀代数表达式中的**括号（圆括号、中括号、大括号）**是否平衡
- 将中缀表达式转换为后缀表达式
- 求值后缀表达式

- 求值中缀表达式

括号匹配

```
1 public static boolean isBalanced(String string) {
2     Stack<Character> stack = new Stack<Character>();
3     for (int i = 0; i < string.length(); i++) {
4         switch (string.charAt(i)) {
5             case '{'->{
6                 stack.push('{'); // 左括号统一入栈
7             }
8             case '}'->{
9                 if(stack.peek()=='{') // 右括号则检查栈顶是否是对应左括号，是的话弹出
10                    stack.pop();
11            }
12            case '['->{
13                stack.push '[';
14            }
15            case ']'->{
16                if(stack.peek()=='[')
17                    stack.pop();
18            }
19            case '('->{
20                stack.push('(');
21            }
22            case ')'->{
23                if(stack.peek()=='(')
24                    stack.pop();
25            }
26        }
27    }
28    return stack.isEmpty();
29 }
```

后缀表达式

*你需要直到怎么用草稿纸把中序表达变成后续表达即可。

中缀表达式和后缀表达式是数学表达式的两种不同表示方式。

中缀表达式 (Infix Expression) :

中缀表达式是我们通常书写数学表达式的方式，其中运算符位于两个操作数之间。例如：

$(a + b) * c$

后缀表达式 (Postfix Expression) :

后缀表达式（也称为逆波兰表达式）中，运算符位于两个操作数之后。例如，上述中缀表达式的后缀表达式形式为：

$ab + c*$

如何将中缀表达式转换为后缀表达式：

可以通过使用栈的数据结构来实现转换。以下是转换的步骤：

1. 初始化一个空栈用于存放运算符，初始化一个空列表用于输出后缀表达式。
2. 从左到右扫描中缀表达式。
3. 如果遇到操作数，直接将其添加到输出列表。
4. 如果遇到左括号"(", 将其压入栈中。
5. 如果遇到右括号")", 则从栈中弹出元素并添加到输出列表，直到遇到左括号为止。左括号不添加到输出列表，而是从栈中弹出。
6. 如果遇到运算符，则：
 - 假设当前遇到的字符 `currentChar` 和 `stack` 顶部的字符 `peek`。重复以下步骤：
 - 如果 `currentChar` 和 `peek` 是同级的（加减同级，乘除同级），弹出 `peek`，输出列表添加 `peek`
 - 如果 `currentChar` 级别小于 `peek`，弹出 `peek`，输出列表添加 `peek`
 - 直到遇到 `peek` 级别比自己小，或者遇到 `(`，或者 `stack.isEmpty()==true`
7. 扫描完中缀表达式后，将栈中剩余的运算符依次弹出并添加到输出列表。
8. 输出列表即为后缀表达式。

示例：

将中缀表达式 $(a + b) * c$ 转换为后缀表达式：

1. 初始状态：栈 = [], 输出列表 = []
2. 遇到 '(', 压入栈中：栈 = ['('], 输出列表 = []
3. 遇到 'a', 添加到输出列表：栈 = ['('], 输出列表 = ['a']
4. 遇到 '+', 压入栈中：栈 = ['(', '+'], 输出列表 = ['a']
5. 遇到 'b', 添加到输出列表：栈 = ['(', '+'], 输出列表 = ['a', 'b']
6. 遇到 ')', 弹出栈中元素 '+' 并添加到输出列表，弹出 '('：栈 = [], 输出列表 = ['a', 'b', '+']
7. 遇到 '*', 压入栈中：栈 = ['*'], 输出列表 = ['a', 'b', '+']
8. 遇到 'c', 添加到输出列表：栈 = ['*'], 输出列表 = ['a', 'b', '+', 'c']
9. 扫描完表达式，弹出栈中剩余的 '*' 并添加到输出列表：输出列表 = ['a', 'b', '+', 'c', '*']

最终的后缀表达式为： $ab + c*$

后缀表达式更容易处理：后缀表达式的计算顺序是固定的，不需要考虑运算符的优先级和括号的影响

练习：

写出这几个中缀表达式的对应后缀表达式

```
1 System.out.println(compile("{3+[d-7*(g+5)]/w} "));
2 System.out.println(compile("(a+n)*(b-8*m)"));
3 System.out.println(compile("b/v^7"));
4 System.out.println(compile("[ (4+b) -2 ]"));
```

答案：

```
1 3d7g5+*-w/+
2 an+b8m*-~
3 bv7^/
4 4b+2-
```

非PPT内容

代码实现如下:

```
1 public class InfixToPostfix {
2     public static void main(String[] args) {
3         System.out.println(compile("{3+[d-7*(g+5)]/w} "));
4         System.out.println(compile(" (a+n)*(b-8*m)"));
5         System.out.println(compile("b/v^7"));
6         System.out.println(compile("[ (4+b)-2 ]"));
7     }
8
9
10    public static String compile(String string) {
11        finalResult = new StringBuilder();
12        stack = new Stack<>();
13        string = deleteSpace(string); // 删除空格
14        string = fixBracket(string); // 把[, {, }, ]这些括号转为()
15        if(!Balance.isBalanced(string))// 检测这个String是否括号匹配上了
16            System.err.println("Bracket is not balanced!");
17
18        for (int i = 0; i < string.length(); i++) {
19            char currentChar = string.charAt(i);
20            switch (currentChar) {
21                case '+', '-', '*', '/', '^' -> {
22                    charOperator(currentChar);
23                }
24                case '(', ')' -> {bracketOperator(currentChar);}
25                default -> finalResult.append(currentChar);
26            }
27        }
28        popAll();
29        return finalResult.toString();
30    }
31
32
33    private static StringBuilder finalResult = new StringBuilder();
34
35    private static Stack<Character> stack = new Stack<>();
36
37    private static String fixBracket(String string) {
38        StringBuilder result = new StringBuilder(string);
39        for (int i = 0; i < result.length(); i++) {
40            switch (result.charAt(i)) {
41                case '[', '{' -> result.setCharAt(i, '(');
42                case ']', '}' -> result.setCharAt(i, ')');
```



```

43     }
44 }
45     return result.toString();
46 }
47
48 private static void popAll() {
49     while (!stack.isEmpty()) {
50         finalResult.append(stack.pop());
51     }
52 }
53
54 private static void charOperator(char operator) {
55     if(stack.isEmpty()){ // 如果栈空，直接放入
56         stack.push(operator);
57         return;
58     }
59     // 栈不为空，一直弹出优先级比自己高的，也就是要找到 popOrNot == false 的
60     while(!stack.isEmpty() && popOrNot(operator, stack.peek()))
61     {
62         finalResult.append(stack.pop());
63     }
64     stack.push(operator); // 放入Operator
65 }
66 /*如果 peek < currentChar （优先级） 无需弹出
67 如果 peek >= currentChar （优先级） 需要弹出
68
69 例如 peek = * < currentChar = ^ 无需弹出
70 例如 peek = * >= currentChar = + 需要弹出*/
71 private static boolean popOrNot(char currentChar, char peek) {
72     if(peek == '(') return false; // 特殊，顶级优先级
73     // 同级别弹出
74     if(currentChar == peek) return true; // 特殊，相等要弹出（包括同类型^）
75     if((currentChar=='*' || currentChar=='/') && (peek=='*' || peek=='/')) return
true; // 同类型乘除法弹出
76     if((currentChar=='+' || currentChar=='-') && (peek=='+' || peek=='-')) return
true; // 同类型加减法弹出
77
78     // 不同级别
79     if(currentChar == '^') return false; // current 是最高优先级的^，不弹出
80     if(peek == '^') return true; // peek是最高优先级的^，直接弹出
81
82     // 现在current和peek 只能是 +-* / 之一
83     if(currentChar == '*' || currentChar == '/') return false; // current是乘除
法，无需弹出
84     if((currentChar == '+' || currentChar=='-') && (peek=='*' || peek=='/')) return
true; // peek是乘除法，弹出
85
86
87     System.err.println("STACK ERROR. Current Char [" +currentChar+"] Compare Char
["+peek+"]");
88     return false;
89 }
90 // 处理括号

```

```

91 private static void bracketOperator(char currentChar) {
92     if(currentChar == '('){
93         stack.push(currentChar);
94         return;
95     }
96     // 剩下的是 ")"
97     while(!stack.isEmpty() && stack.peek() != '(')
98         finalResult.append(stack.pop());
99     if(!stack.isEmpty() && stack.peek() == '(')
100         stack.pop();
101 }
102 // 删除空格
103 private static String deleteSpace(String string) {
104     if (string == null || string.isEmpty()) return "";
105     StringBuilder finalResult = new StringBuilder(string);
106     for(int i = 0; i < finalResult.length(); i++) {
107         while (i < finalResult.length() && finalResult.charAt(i) == ' ')
108             finalResult.deleteCharAt(i);
109     }
110     return finalResult.toString();
111 }
112 }
113
114 }

```

[5] 图 Map

数据的集合，但不是单个值的集合：

- 映射（Map）= 键到值的对的集合
- 受限的访问：通过键获取值
- 不允许重复的键
- 有多种实现，最常见的是哈希映射（HashMap）

在声明和构造时，必须指定两种类型：

- **键的类型和值的类型**

```

1 private Map<String, Integer> phoneBook; // K, V
2 phoneBook = new HashMap<String, Integer>(); // K, V

```

- 核心操作：
 - `get(key)`，→ 返回与键关联的值（如果不存在则返回 null）
 - `put(key, value)`，→ 设置与键关联的值（并返回旧值，如果有的话）
 - `remove(key)`，→ 移除键及其关联的值（并返回旧值，如果有的话）
 - `containsKey(key)`，→ 返回布尔值
 - `size()` 返回大小

应用

字数统计：给出一个文本，查找哪个文字的出现频率最高。这里我们把“文本”定义为一个 `char`。我们尝试用 `Map` 来实现

```
1 public class Frequency {
2     public static void main(String[] args) {
3         String s = "Once upon a time, in a small village, a curious cat named Max often
4         wandered the streets. One day, he found a shiny key near an old oak tree. Intrigued,
5         Max carried the key home. His owner, Mia, recognized it as the key to her grandmother's
6         antique chest. Inside, they discovered a treasure map. Together, Max and Mia embarked
7         on an adventure to find the hidden treasure, learning the true value of curiosity and
8         teamwork along the way.";
9         compile(s);
10        getFrequency();
11    }
12    private static Map<Character,Integer> map = new HashMap<>();
13    public static void compile(String string){
14        StringBuilder sb = new StringBuilder(string);
15        for (int i = 0; i < sb.length(); i++) {
16            char currentChar = sb.charAt(i);
17            if(!map.containsKey(currentChar)) // 如果这个Map不存在这个char，就存入
18                map.put(currentChar,1);
19            else
20                map.put(currentChar,map.get(currentChar)+1); // 如果存在，就把Value ++;
21        }
22    }
23    public static void getFrequency(){
24        StringBuilder sb = new StringBuilder();
25        int maxVal = 0;
26        char key='0';
27        for( Map.Entry<Character,Integer> entry : map.entrySet()){
28            if(entry.getValue() > maxVal)
29            {
30                maxVal = entry.getValue();
31                key = entry.getKey();
32            }
33        }
34        for( Map.Entry<Character,Integer> entry : map.entrySet()){
35            System.out.println("[Char :'" +entry.getKey()+"\'"+ " Used time :'"
36            +entry.getValue()+"'"]");
37        }
38    }
39 }
```

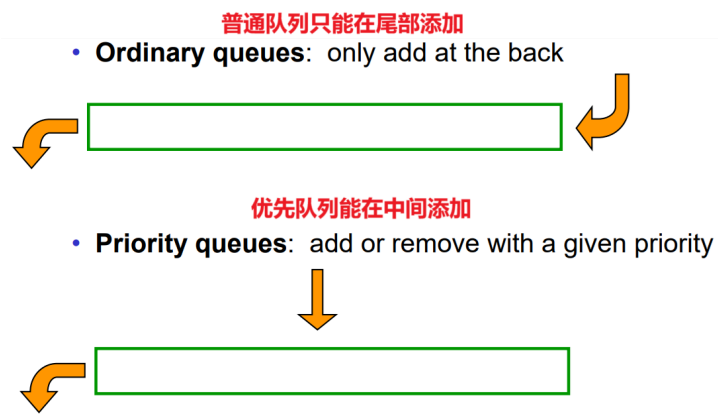
```
1 [Char : 'H' Used time : '1']
2 [Char : 'I' Used time : '2']
3 [Char : 'M' Used time : '5']
4 [Char : 'O' Used time : '2']
5 [Char : 'T' Used time : '1']
6 [Char : '' Used time : '1']
7 [Char : ' ' Used time : '77']
8 [Char : 'a' Used time : '36']
9 [Char : 'b' Used time : '1']
10 [Char : 'c' Used time : '8']
11 [Char : 'd' Used time : '20']
12 ...
```

■ [Week 4 Lecture 5]

[1] 队列

队列与栈相似但是不同

1. 队列是具有**顺序**的值的集合
2. 受限的访问：只能从前面移除
3. 两种类型：
 1. 普通队列：只能在尾部添加
 2. 优先队列：根据给定的优先级添加或移除



用途

- 操作系统、网络应用、多用户系统
- 处理必须按顺序完成的请求/事件/任务（在这种上下文中，通常将存储此类请求的内存池称为“缓冲区”）
- 模拟程序
- 表示现实世界中的队列（交通、客户、送货等）
- 管理将来必须发生的事件
- 搜索算法
- 计算机游戏

- 人工智能
- Java提供：一个 `Queue` 接口，多个类： `LinkedList`、 `PriorityQueue`

队列操作

- `offer(value)` $\Rightarrow$ `boolean`
 - 将值添加到队列中
 - (有时称为“入队”)
- `poll()` $\Rightarrow$ `value`
 - 移除并返回队列前端/头部的值，如果队列为空则返回 `null`
 - (有时称为“出队”，类似于“弹出”)
- `peek()` $\Rightarrow$ `value`
 - 返回队列头部的值，如果队列为空则返回 `null`
(不会从队列中移除)
- `remove()` 和 `element()`
 - 类似于 `poll()` 和 `peek()`，但如果队列为空则会抛出异常。

使用标准 for-each 循环遍历

```
1  for (Face face : crowd) {
2      face.render(canvas);
3  }
4  for (Map.Entry<String,Integer> entry : phonebook.entrySet()){
5      textArea.append(entry.getKey()+" : "+entry.getValue());
6  }
```

为什么需要迭代器？

- 程序无法直接访问集合 `Collection` 对象的内部
- 集合对象会构造一个迭代器
- 迭代器可以访问集合的内部
- 迭代器逐个提供元素
- 每个集合类都需要一个关联的迭代器类

[2] 迭代器

1. 迭代器和可迭代对象的基础概念

- `Iterable` 接口要求实现类提供一个 `Iterator`。
- `Iterator` 接口要求实现类提供 `hasNext()` 和 `next()` 方法。
- 例如，所有集合 (`Collections`) 都实现了 `Iterable` 接口。

2. 迭代器的使用

- 迭代器的操作：
 - `hasNext()`：如果还有值可以获取，则返回 `true`
 - `next()`：返回下一个值
- 标准使用模式：

```

1  Iterator<type> itr = construct iterator; // 构造迭代器
2  while (itr.hasNext()) {
3      type var = itr.next(); // 获取下一个值
4      ... var ... // 使用该值
5  }

```

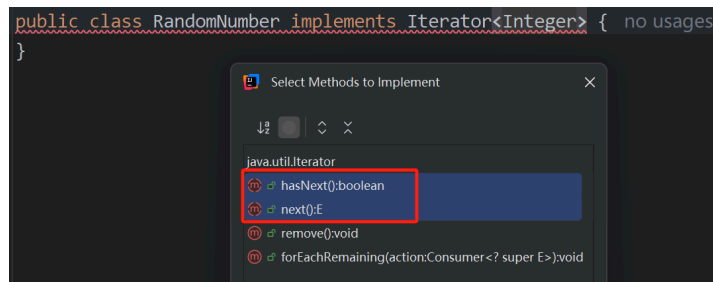
- 几乎与“for each”循环相同：

```

1  for (type var : collection) {
2      ... var ... // 使用集中的每个值
3  }

```

3. 创建迭代器



要实现 `Iterator<E>` 方法必须 `@Override` 这两个方法

1. `hasNext()`，返回 `Boolean`
2. `next()`，返回泛型 `E`

在下面这个例子中，我们把整型当成 `E`。

- 迭代器不仅用于集合对象。
- 可以用于任何生成值序列的对象，例如 `Scanner` 或伪随机数生成器。
- 伪随机数生成器的示例：

```

1  public class NumCreator implements Iterator<Integer> {
2      private int num = 1;
3      public boolean hasNext() {
4          return true;
5      }
6      public Integer next() {
7          num = (num * 92863) % 104729 + 1;
8          return num;
9      }
10 }

```

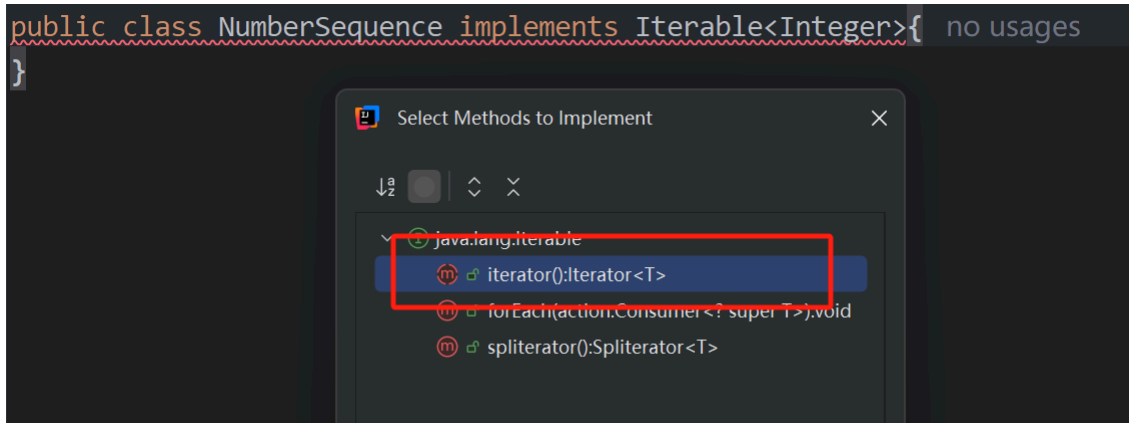
- 使用示例：

```

1  Iterator<Integer> lottery = new NumCreator();
2  for (int i = 1; i < 1000; i++)
3      textArea.append(lottery.next() + "\n");

```

4. 创建可迭代对象



要实现 `Iterable<E>` 方法必须 `@Override` 一个方法

1. `iterator()`，返回一个迭代器对象 `Iterator<E>`

• 例如：`NumberSequence` 类表示一个无限的算术序列，包含起始数字和步长，例如 5, 8, 11, 14, 17,....

```
1  ``java
2  public class NumberSequence implements Iterable<Integer> {
3      private int start;
4      private int step;
5      public NumberSequence(int start, int step) { // 构造方法
6          this.start = start;
7          this.step = step;
8      }
9      public Iterator<Integer> iterator() { // 返回Iterator<Integer>型迭代器对象
10
11          // 传入 NumberSequence 这个可迭代对象
12          return new NumberSequenceIterator(this);
13      }
14  }
15  ``
```

接下来我们来创建 `NumberSequenceIterator`

5. 为可迭代对象 `NumberSequence` 创建迭代器

- `NumberSequenceIterator` 类实现了 `Iterator` 接口。

```
1  // 可迭代对象 NumberSequence
2  public class NumberSequence implements Iterable<Integer> {
3      private int start;
4      private int step;
5      public NumberSequence(int start, int step) { // 构造方法
6          this.start = start;
7          this.step = step;
8      }
9      public Iterator<Integer> iterator() {
10         return new NumberSequenceIterator(this);
11     }
12     // 内部类，迭代器NumberSequenceIterator
13     private class NumberSequenceIterator implements Iterator<Integer> {
14         private int nextNum;
```

```

15     private NumberSequence source;
16     public NumberSequenceIterator(NumberSequence ns) {
17         source = ns; // 赋值，获取到NumberSequence这个可迭代对象
18         nextNum = ns.start;
19     }
20     public boolean hasNext() {
21         return true;
22     }
23     public Integer next() {
24         int ans = nextNum;
25         nextNum += source.step;
26         return ans;
27     }
28 }
29 }

```

6. 使用可迭代对象

- 可以在“for each”循环中使用可迭代对象：

```

1  for (int n : new NumberSequence(15, 8)) {
2      System.out.printf("next number is %d \n", n);
3  }

```

- 也可以直接使用可迭代对象的迭代器：

```

1  Iterator<Integer> iter = new NumberSequence(15, 8).iterator();
2  processFirstPage(iter);
3  for (int p = 2; p < maxPages; p++)
4      processNextPage(p, iter);

```

总结：这段内容展示了如何使用迭代器和可迭代对象来处理序列数据，并提供了具体的代码示例来演示如何创建和使用它们。

[3] Q&A

- Java has specified a “Queue” interface. (T or F)

- Java 是否定义了“Queue”接口。 (T 或 F)

答案：T

Java 确实定义了 Queue 接口。

- Java does not have any class support for “Priority Queue”. (T or F)

- Java 是否没有提供任何支持“Priority Queue”的类。 (T 或 F)

答案：F

Java 提供了 PriorityQueue 类来支持优先队列。

- peek() operation under the Queue interface will throw an exception if the queue is empty. (T or F)

- 如果队列为空，Queue 接口中的 peek() 操作是否会抛出异常。 (T 或 F)

答案：F

peek() 操作在队列为空时返回 null，不会抛出异常。

- poll() operation under the Queue interface will throw an exception if the queue is empty. (T or F)

- 如果队列为空，`Queue` 接口中的 `poll()` 操作是否会抛出异常。 (T 或 F)

答案: F

`poll()` 操作在队列为空时返回 `null`，不会抛出异常。

- There is an `element()` method under the `Queue` interface. (T or F)

- `Queue` 接口中是否有一个 `element()` 方法。 (T 或 F)

答案: T

`Queue` 接口确实定义了 `element()` 方法，它在队列为空时会抛出 `NoSuchElementException`。

- `Iterable` is an interface specification for a class that is equipped with an `Iterator`.

- `Iterable` 是一个接口规范，用于具有迭代器的类。

答案: T

`Iterable` 接口要求实现类提供一个 `Iterator`。

- `Iterator` is an interface specification for a class that can generate iterative elements.

- `Iterator` 是一个接口规范，用于可以生成迭代元素的类。

答案: T

`Iterator` 接口要求实现类提供 `hasNext()` 和 `next()` 方法，用于生成迭代元素。

■ [Week 4 Lecture 6]

排序集合

[1] 两种排序方式

第一种方法，默认排序/自然排序

让 `Student` 类实现 `Comparable` 接口。

```
1 public class student implements Comparable<student>{
2     @Override
3     public int compareTo(student o) {
4         return this.age - o.age;
5     }
6 }
```

第二种方法：创建比较器对象（匿名内部类）

```
1 TreeSet<String> ts = new TreeSet<>(new Comparator<String>() {
2     @Override
3     public int compare(String o1, String o2) {
4         if(o1.length() > o2.length()) return 1;
5         return o2.compareTo(o1);
6     }
7 });
```

如果两种方法同时存在，以方式二为准（例如 `String` 下定义了比较方法，但是仍然可以用比较器 `Comparator` 来比较）。

例子：

```

1 public static void main(String[] args) {
2     int[] arr1 = {2, 1, 4, 3, 5};
3     // 升序排列（默认）
4     Arrays.sort(arr1);
5     System.out.println(Arrays.toString(arr1));
6     // 降序排列
7     Integer[] arr2 = {2, 1, 4, 3, 5};
8     Arrays.sort(arr2, new Comparator<Integer>() {
9         @Override
10         public int compare(Integer o1, Integer o2) {
11             return o2 - o1;
12         }
13     });
14     System.out.println(Arrays.toString(arr2));
15     // 降序排列（用Lambda表达式简化）
16     arr2 = new Integer[]{2, 1, 4, 3, 5};
17     Arrays.sort(arr2, (Integer o1, Integer o2) -> {
18         return o2-o1;
19     });
20     System.out.println(Arrays.toString(arr2));
21 }

```

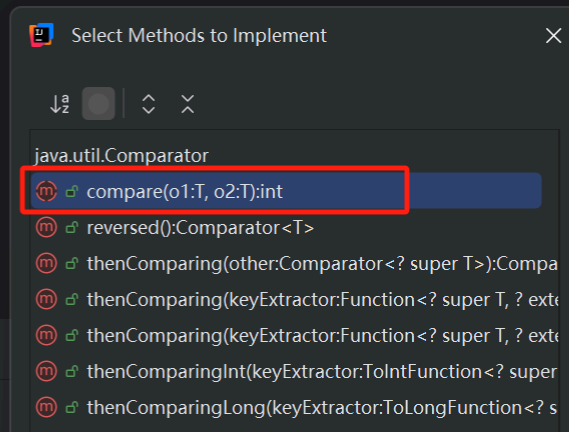
除了在类中定义，或者使用匿名内部类的方式，我们还可以自己创建一个Compare类（通过实现Compare接口）

要实现这个类必须重写 `compare` 方法

```

private class customizeComparator implements Comparator<Integer> { no usages
}

```



```

1 public static void main(String[] args) {
2     ArrayList<Integer> list = new ArrayList<Integer>();
3     list.add(3);
4     list.add(1);
5     list.add(4);
6     list.add(5);
7     list.add(2);
8
9     System.out.println(list);
10    Collections.sort(list, new customizeComparatorIncrease());
11    System.out.println(list);

```

```

12     Collections.sort(list, new customizeComparatorDecrease());
13     System.out.println(list);
14 }
15 private static class customizeComparatorIncrease implements Comparator<Integer> {
16
17     @Override
18     public int compare(Integer o1, Integer o2) {
19         return o2-o1; // 降序
20     }
21 }
22 private static class customizeComparatorDecrease implements Comparator<Integer> {
23
24     @Override
25     public int compare(Integer o1, Integer o2) {
26         return o1-o2; // 降序
27     }
28 }

```

输出如下

```

1  [3, 1, 4, 5, 2]
2  [5, 4, 3, 2, 1]
3  [1, 2, 3, 4, 5]

```

确实可以通过自定义比较器来进行比较。

`Comparable` 和 `Comparator` 接口的区别和用法：

1. `Comparable` 接口：

- 类应该实现 `Comparable` 接口来控制它们的自然排序。
- 实现 `Comparable` 的对象可以通过 `Collections.sort()` 和 `Arrays.sort()` 进行排序，并且可以作为排序映射中的键或排序集中的元素，而无需指定 `Comparator`。
- `compareTo(Object)` 方法用于将此对象与另一个对象进行比较，返回负整数、零或正整数，分别表示此对象小于、等于或大于另一个对象。

2. `Comparator` 接口：

- 使用 `Comparator` 可以按照非自然顺序对对象进行排序。
- `compare(Object, Object)` 方法用于比较两个参数的顺序，返回负整数、零或正整数，分别表示第一个参数小于、等于或大于第二个参数。

总的来说，`Comparable` 用于定义对象的自然排序，而 `Comparator` 用于定义自定义的排序规则。

[2] Q&A

1. An object defined under a comparable class will have a “natural ordering”. (T or F)

翻译：一个定义在 `Comparable` 类下的对象将会具有“自然排序”。（正确或错误）

True

解释：一个类实现 `Comparable` 接口后，它的对象就具有“自然排序”，因为 `Comparable` 接口定义了 `compareTo` 方法。

2. Objects declared under a comparable class can be compared using which method?

翻译：声明在 `Comparable` 类下的对象可以使用哪个方法进行比较？

`compareTo` 方法

解释：`Comparable` 接口中的 `compareTo` 方法用于比较对象。

3. What is the signature of the compareTo method?

翻译：`compareTo` 方法的签名是什么？

`int compareTo(T o)`

解释：`compareTo` 方法的签名为 `int compareTo(T o)`，其中 `T` 是泛型类型。

4. Which method can be used to sort list of comparable objects?

翻译：哪个方法可以用来排序 `Comparable` 对象的列表？

`Collections.sort()`

解释：`Collections.sort()` 方法可以用来排序实现 `Comparable` 接口的对象的列表。

5. Comparator is an object that can compare other objects. (T or F)

翻译：`Comparator` 是一个可以比较其他对象的对象。（正确或错误）

True

解释：`Comparator` 是一个接口，它可以用于比较两个对象，因此它是可以比较其他对象的对象。

6. What is the signature for the compare() method?

翻译：`compare()` 方法的签名是什么？

`int compare(T o1, T o2)`

解释：`compare()` 方法的签名为 `int compare(T o1, T o2)`，其中 `T` 是泛型类型。

7. A comparable class can implement multiple comparators. (T or F)

翻译：一个 `Comparable` 类可以实现多个 `Comparator`。（正确或错误）

True

解释：一个 `Comparable` 类可以实现多个 `Comparator`，因为 `Comparator` 是独立的接口，可以定义多种比较逻辑。

■ [Week 4 Lecture 7]

异常 Exception

[1] 异常处理 (Exceptions)

这段内容主要讲解了异常的概念、抛出异常的原因、异常的类型以及如何在代码中捕获和处理异常。

1. 什么是异常？

- 异常是程序执行过程中发生的事件，它会中断程序的正常指令流。

- 异常是一种“非局部退出”，这意味着它会打断当前的执行流程并跳转到异常处理逻辑。

2. 何时会抛出异常？

- 发生某种错误时，例如：
 - 文件不存在
 - 除以零
 - 在空对象上调用方法
 - 调用未定义的方法
- 出现异常情况时，例如：
 - 文件中的文本不符合预期格式
 - 用户输入了意外的答案

3. 如何处理异常？

- 抛出异常 (`throw`)
- 捕获异常 (`catch`)

4. 异常的来源

- JVM 在执行代码时抛出
- Java 库中的代码抛出
- 开发者的代码主动抛出

5. 捕获异常

使用 `try...catch` 语句捕获异常

例如，我们写了一个能够写入文件的方法：

```
1 public static void write(String path, String text, int mode){
2     boolean append = false;
3     if(mode == OVERRIDE_ALL) append = false;
4     if(mode == APPEND) append = true;
5
6     try {
7         writer = new BufferedWriter(new FileWriter(path, append));
8         writer.write(text);
9         writer.close();
10    } catch (IOException e) {
11        throw new RuntimeException(e);
12    }
13 }
```

在这里就有捕获 `IOException` 的 `try-catch` 方法了

异常类型

- `RuntimeException` 不需要处理：
 - 未捕获的 `RuntimeException` 会导致错误信息
 - 如果你愿意，可以捕获它们。
- 其他异常必须处理：

- 例如 `IOException`
(这就是为什么我们在打开文件时总是使用 `try...catch`)。
- 异常对象包含信息：
 - 执行栈的状态
 - 任何与异常相关的额外信息

[2] Q&A

1. A method will do what when something goes wrong?

翻译：当出现问题时，方法会做什么？

答案：方法会抛出异常（`throw an exception`）。

2. An exception provides a local exit when something goes wrong. (T or F)

翻译：异常在出现问题时提供了一种局部退出的方式。（正确或错误）

True

解释：异常是一种机制，可以在出现错误时立即停止当前代码的执行，并跳转到异常处理代码。

3. Name 3 cases when an exception can be thrown.

翻译：列举3种可能抛出异常的情况。

答案：

- 访问数组时下标越界（`ArrayIndexOutOfBoundsException`）。
- 除零操作（`ArithmeticException`）。
- 访问空对象的方法或属性（`NullPointerException`）。

4. What do we do with exceptions?

翻译：我们如何处理异常？

答案：我们可以捕获异常（`catch`）并处理它，或者将异常抛出（`throw`）给调用者。

5. Exceptions can be caught using what Java statement?

翻译：异常可以使用哪种Java语句捕获？

答案：使用 `try-catch` 语句。

6. Does exception have a type?

翻译：异常是否有类型？

答案：是，异常是 `Throwable` 的子类，例如 `Exception` 和 `RuntimeException`。

7. After the exception handler finishes its work, what will the program do next?

翻译：异常处理程序完成工作后，程序接下来会做什么？

答案：程序会继续执行 `try-catch` 块之后的代码。

8. Can exception handler use information in the exception object?

翻译：异常处理程序可以使用异常对象中的信息吗？

答案：是，例如通过 `getMessage()` 方法获取异常的详细信息。

9. What does “e.getMessage()” do where e is an exception object?

翻译：其中 `e` 是异常对象，“e.getMessage()”会做什么？

答案：返回异常的详细信息字符串。

10. Name 3 common Java exceptions.

翻译：列举3种常见的Java异常。

答案：

- `NullPointerException`

- `ArrayIndexOutOfBoundsException`
- `IOException`

11. `RuntimeExceptions` doesn't have to be handled. (T or F)

翻译: `RuntimeException` 不需要被处理。(正确或错误)

True

解释: `RuntimeException` 是 `unchecked exception`, 不强制要求捕获或声明。

12. `IOException` doesn't have to be handled. (T or F)

翻译: `IOException` 不需要被处理。(正确或错误)

False

解释: `IOException` 是 `checked exception`, 必须捕获或声明抛出。

13. Which Java statement can cause an exception?

翻译: 哪种Java语句可能导致异常?

答案: `throw` 语句。

14. What happens when we try adding an element to an immutable List?

翻译: 当我们尝试向一个不可变的 `List` 添加元素时会发生什么?

答案: 抛出 `UnsupportedOperationException`。

15. Does a Java Interface provide a 'constructor'?

翻译: Java接口是否提供“构造函数”?

答案: 否, 接口不能包含构造函数。

16. `ArrayList` implements which Interface?

翻译: `ArrayList` 实现了哪个接口?

答案: `List`。

17. `public interface List` extends which Java Interface?

翻译: `public interface List <E>` 继承自哪个Java接口?

答案: `Collection<E>`。

■ [Week 5 Lecture 8,9,10]

实现Collections: `Interfaces`, `Abstract Classes`, `Classes`

介绍了接口和继承(抽象)类

Lecture 8

[1] `ArrayList` 设计思路

`List` 是 Java 中的一个接口, 它定义了很多方法, 比如添加元素、获取元素、删除元素、判断是否包含某个元素等等。`ArrayList` 是实现 `List` 接口的一个类, 这就意味着 `ArrayList` 必须实现 `List` 接口中定义的所有方法。但是 `List` 接口中的方法数量很多, 而且其中不少方法的实现逻辑比较复杂, 这给 `ArrayList` 类的编写带来了困难。

解决思路：

虽然List接口中的方法很多，但其实许多复杂方法可以通过几个基本的方法来实现。这里提到的基本方法包括size（用于获取集合中元素的数量）、add（用于向集合中添加一个元素）、get（用于根据索引获取集合中的元素）、set（用于根据索引设置集合中某个位置的元素）、remove（用于移除集合中的某个元素）。

先看构造：

```
1 public class ArrayList <E> implements List<E> {
2     private E [ ] data; // 泛型数组数据
3     private int count; // 记录数量
4     ....
5 }
```

虽然有些方法很复杂，例如 addAll()，但是可以通过调用自身的**基本方法**来实现。例如调用基本方法 add()：

```
1 public boolean addAll(Collection<E> other){
2     for (E item : other) // 增强for遍历other然后利用this.add()这个自身方法添加...
3         add(item);
4 }
```

实际上的ArrayList：

```
1 public class ArrayList<E> extends AbstractList<E>
2     implements List<E>, RandomAccess, Cloneable, java.io.Serializable
3 {
4     ...
5 }
```

先看继承的抽象类

抽象类里既可以有抽象方法（无方法体），也可以有普通方法（有方法体）。

可以看到ArrayList继承了AbstractList这个抽象类

然后AbstractList里面就写了一些简单的方法（有方法体），当然也有抽象方法（无方法体）。

为何要让AbstractList设计为抽象类？因为它需要强制（继承他的）子类复写所有抽象方法，不然自身的一些方法就不可用，例如

```
1 // AbstractList
2 public abstract E get(int index);
3 public abstract int size();
4 // 无方法体
```

然而，在AbstractList中，有一些方法依赖get()方法，例如Iterator()


```

1 // AbstractList
2 public Iterator<E> iterator() {
3     return new Itr();
4 }

```

```

1 // AbstractList
2 private class Itr implements Iterator<E> {
3     ...
4
5     public boolean hasNext() {
6         return cursor != size(); // 这里用到了
7     }
8
9     public E next() {
10        ...
11        try {
12            ...
13            E next = get(i); // 这里用到了
14            ...
15        } catch (IndexOutOfBoundsException e) {
16            ...
17        }
18    }
19
20 }

```

再看继承的接口

接口中的方法都是抽象方法，没有方法体。在 Java 8 之前，接口只能包含抽象方法；Java 8 及以后，接口可以包含默认方法和静态方法，它们有具体的方法体。

一个类可以实现多个接口，这打破了类的单继承限制，让类可以具备多种不同的行为。

接口里的变量默认是 `public static final` 类型的，也就是常量，必须在定义时就进行初始化，并且之后不能再修改。

接口的抽象方法和默认方法

```

1 interface Vehicle {
2     // 抽象方法
3     void start();
4     // 默认方法
5     default void honk() {
6         System.out.println("Vehicle is honking!");
7     }
8 }
9 // 子类不重写 honk 方法，将使用接口中的默认实现

```

接口的静态方法

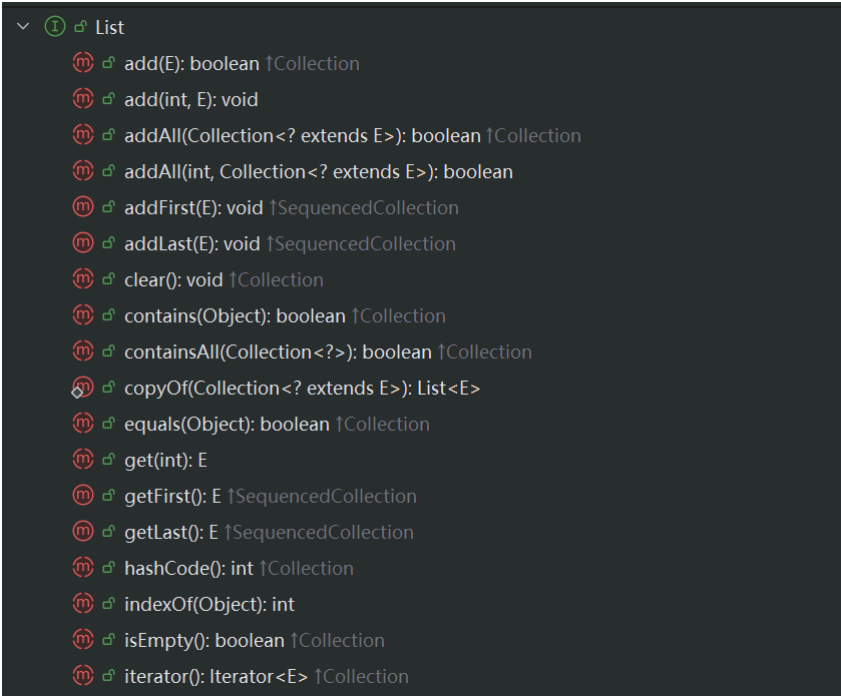
使用 `static` 关键字修饰，在接口中提供静态的实现。静态方法属于接口本身，不能通过实现类的实例来调用，只能通过接口名直接调用。

```
1 interface MathUtils {
2     // 静态方法，计算两个数的和
3     static int add(int a, int b) {
4         return a + b;
5     }
6 }
```

好了，现在我们已经了解完接口，让我们看看ArrayList继承了什么接口！

```
1 public class ArrayList<E> extends AbstractList<E>
2     implements List<E>, RandomAccess, Cloneable, java.io.Serializable
3 {
4     ...
5 }
```

最明显的时它继承了 `List<E>` 这个接口



可以发现实际上List接口都是做了一些规范，然后让实现类去复写一般抽象方法，当然也有static方法，这些方法只能调用接口使用。 -

类别	描述	示例（以Java集合相关为例）
Interface	指定类型；定义方法头	<code>List <E></code> ：指定E类型的序列
Abstract Class	实现接口；定义部分方法，将其他方法设为“抽象”	<code>AbstractList <E></code> ：实现 <code>List <E></code> 接口；定义E类型数组；定义 <code>addAll</code> 、 <code>subList</code> 等方法； <code>add</code> 、 <code>set</code> 、 <code>get</code> 等方法为抽象方法
Class	继承抽象类；定义数据结构、基本方法和构造函数	<code>ArrayList <E></code> ：继承 <code>AbstractList</code> ；实现字段和构造函数；实现 <code>add</code> 、 <code>get</code> 等方法

关于泛型数组

泛型数组不允许直接创建。比如我们现在想要创建一个 `E[]`

```
public class MyList<E> { no usages new *
    private E[] data; 1 usage
    private final int size = 16; // 固定值 1 usage
    public MyList() { no usages new *
        data = new E[size];
    }
}
```

这是不允许的。但是我们可以创建一个 `Object[]` 然后再用类型转换转为 `E[]`

```
public class MyList<E> { no usages new *
    private E[] data; 1 usage
    private final int size = 16; // 固定值 1 usage
    public MyList() { no usages new *
        data = (E[]) new Object[size];
    }
}
```

Unchecked cast: 'java.lang.Object[]' to 'E[]'

此时编译器会弹出警告：未检查的映射，为了“隐藏”这个问题，可以加上：

```
public class MyList<E> { no usages new *
    private E[] data; 1 usage
    private final int size = 16; // 固定值 1 usage
    @SuppressWarnings("unchecked") no usages new *
    public MyList() {
        data = (E[]) new Object[size];
    }
}
```

© java.lang.Object

@Contract(pure = true) ↗
public Object()

[2] ArrayList中的一些简单方法

set方法

```
1 public E set(int index, E value){
2     if (index < 0 || index >= count)
3         throw new IndexOutOfBoundsException();
4     E ans = data[index];
5     data[index] = value;
6     return ans; // ans时弹出值,就是旧值。如果没有则是默认值
7 }
```

remove方法：

```

1 public E remove (int index){
2     if (index < 0 || index >= count)
3         throw new IndexOutOfBoundsException();
4     E ans = data[index]; // 旧值
5     for (int i=index+1; i< count; i++) // 这里就体现了ArrayList的删除过程巨慢
6         data[i-1]=data[i];
7     count--; // 数量-1
8     return ans;
9 }

```

注意! 现在我们要删除index, 那么下面哪种删除方法是正确的?

```

1 for (int i=index; i< count; i++)
2     data[i]=data[i+1];

```

```

1 for (int i=index+1; i< count; i++)
2     data[i-1]=data[i];

```

第一种会有**数组越界**风险, 因为 `i==count-1` 没问题但是 `i+1 == count` 可能导致数组越界

add 方法

```

1 public void add(int index, E item){
2     if (index < 0 || index > count)
3         throw new IndexOutOfBoundsException(); // 如果用户输入的index大于count或者小于0那么就
报错
4     ensureCapacity(); // 这一步很重要, 确保容量足够
5     for (int i=count; i > index; i--) // 这种方法规避了数组越界风险
6         data[i]=data[i-1];
7     data[index]=item;
8     count++;
9 }

```

ensureCapacity 方法

```

1 private void ensureCapacity () {
2     if (count < data.length) return; // 空间足够直接返回
3     E [ ] newArray = (E[]) (new Object[data.length+INITIALCAPACITY]); // 第一种: 直接加
INITIALCAPACITY
4     for (int i = 0; i < count; i++) // 复制到newArray
5         newArray[i] = data[i];
6     data = newArray; // 成员变量data现在储存newArray地址值, 然后旧的data的地址值因为没有指向而被回
收
7 }

```

```

1 // 第二种是
2 E [ ] newArray = (E [ ]) (new Object[data.length * 2]);

```

[3] 课后小问题

[问题 1] What are the key features of an abstract class?

翻译：抽象类的关键特征是什么？

1. 抽象类可以包含抽象方法（abstract methods），这些方法只有声明而没有实现。例如：

```
1 abstract class Shape {  
2     // 抽象方法  
3     public abstract double getArea();  
4 }
```

2. 抽象类不能被实例化，只能被继承。例如，上述 `Shape` 类不能使用 `new Shape()` 来创建对象。
3. 抽象类可以包含具体的方法和成员变量，就像普通类一样。例如：

```
1 abstract class Shape {  
2     protected String color; // 成员变量  
3     public Shape(String color) { // 具体方法，子类可选择性重写  
4         this.color = color;  
5     }  
6     public abstract double getArea(); // 子类必须重写  
7 }
```

在 Java 里，`protected` 成员（包括字段与方法）能够在以下范围内被访问：

1. **同一类内部**：在定义了 `protected` 成员的类内部，该成员可被自由访问。
2. **同一包内**：处于同一包中的其他类，不管这些类是否为该类的子类，都可以访问 `protected` 成员。
3. **不同包的子类**：在不同包中的子类能够访问从父类继承来的 `protected` 成员。不过，子类对象只能通过自身对象来访问这些成员，而不能通过父类对象访问。

[问题 2] Can an abstract class be instantiated?

翻译：抽象类可以被实例化吗？

答案：不可以。但是我们可以使用匿名内部类的方式直接创建对应的类，因此来创建对象。

匿名内部类可以直接实现接口和抽象类。

[问题 3] Abstract methods can be defined within a class to save implementation efforts. (T or F)

翻译：可以在类中定义抽象方法以节省实现工作。（对或错）

答案：True。

就像我们上面讲的，`ArrayList`就实现了`AbstractList`这个抽象类，而且这个抽象类实现了很多方法，比如`Iterator()`。但是同时有必须要求子类重写`get()`和`size()`等抽象方法，这样`AbstractList`才能正常工作。

[问题 4] What are the key issues of implementation when we remove an element from an ArrayList?

翻译：当从 ArrayList 中移除一个元素时，实现的关键问题有哪些？

答案：小心数组越界。然后记得返回旧值。

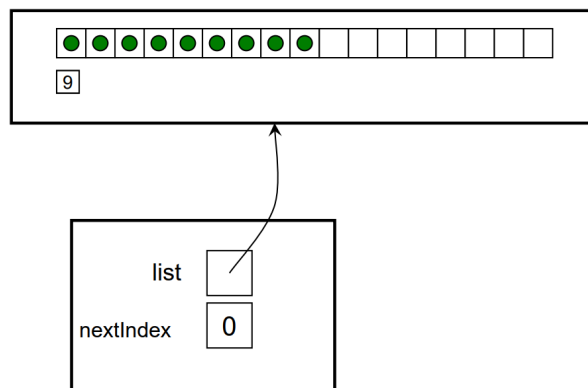
[问题 5] What are the key issues of implementation when we add an element from an ArrayList?

翻译：当向 ArrayList 中添加一个元素时，实现的关键问题有哪些？

答案：检查扩容，大小调整。

Lecture 9

[4] 迭代器 Iterator



基本构造原理

一个对外的类，`Iterator`，当被调用，返回 `ArrayListIterator`。

`ArrayListIterator` 实在 `ArrayList` 中的内部类，因此可以访问 `ArrayList` 的私有成员变量

```
1  /**
2   * 返回一个用于遍历列表中元素的迭代器
3   */
4  public Iterator <E> iterator() {
5      return new ArrayListIterator<E>(this); // 这里传入 arrayList对象
6  }
7
8  /**
9   * 定义用于 ArrayList 的迭代器
10  * 该迭代器定义在 ArrayList 类内部，因此可以访问 ArrayList 对象的私有成员变量
```

```

11  */
12  private class ArrayListIterator <E> implements Iterator <E> {
13      // 成员变量
14      // 构造方法
15      // hasNext() 方法
16      // next() 方法
17      // remove() 方法（迭代器的可选操作）
18  }

```

现在，让我们看看ArrayListIterator到底有什么吧！首先，我们简单看看他是如何能够访问ArrayList的：

```

1  // ArrayListIterator 实现了 Iterator接口
2  // 因为ArrayListIterator是定义在ArrayList内部的类，因此可以访问ArrayList的成员变量
3  private class ArrayListIterator <E> implements Iterator <E>{
4
5      private ArrayList<E> list; // 这个就是待迭代的list
6      private int nextIndex = 0; // 指向下一个index
7      private boolean canRemove = false; // 能否remove
8      private ArrayListIterator (ArrayList <E> list) { // 这里传入 arraylist对象
9          this.list = list;
10     }
11
12     // 是否有下一个元素
13     public boolean hasNext () {
14         return (nextIndex < list.count);
15     }
16     ...
17 }

```

然后，我们再看看它的方法...

next 和 remove方法

```

1  // 返回下一个元素
2  public E next () {
3      if (nextIndex >= list.count) throw new NoSuchElementException();
4      return list.get(nextIndex++); // 比如现在我们第一次获取
5      // 首先，会得到 ArrayList[0]
6      // 接下来nextIndex = 1
7      // return ArrayList[0]而不是1 !!!
8  }
9
10
11 // 该方法的功能是从列表里移除迭代器最后返回的那个元素。
12 public void remove(){
13     throw new UnsupportedOperationException();
14 }
15
16 }

```

这个remove需要更详细地说，就是你只能调用1次（每次next前）

```

1 while (iterator.hasNext()) {
2     Integer number = iterator.next(); // 好，现在迭代器返回了这个元素
3     if (number % 2 == 0) {
4         iterator.remove(); // 这没问题，你删除了现在对应的元素
5     }
6 }

```

```

1 while (iterator.hasNext()) {
2     Integer number = iterator.next();
3     if (number % 2 == 0) {
4         iterator.remove();
5         iterator.remove(); // 这里有问题！你多删除了一次！迭代器根本不知道你要删除谁，因为
    前一个已删除！
6     }
7 }

```

而且，使用迭代器的时候不允许使用 `number.remove()`，因为会带来并发修改异常！（之前讲过）

更详细的remove方法

接下来，我们来讲下他们两是怎么实现的

```

1 /** Return next element in the List */
2 public E next () {
3     if (nextIndex >= list.count) throw new NoSuchElementException();
4     canRemove = true; // 能删除
5     return list.get(nextIndex++); ← increment and return
6 }
7
8 public void remove(){
9     if ( ! canRemove ) throw new IllegalStateException();
10    canRemove = false;
11    nextIndex--; // 这个很重要
12    list.remove(nextIndex);
13 }

```

关于 `nextIndex--`

假设我们现在的ArrayList如下：

Index	0	1	2	3	4
Value	110	123	723	26	10

现在我们用 `Iterator` 来迭代，目标是删除2

现在 `next()` 执行，`canRemove = True`，然后return了 723，接下来 `nextIndex++`，变成了3。

但是我们next完之后想要remove 723,因此调用 `remove` 方法，此时 `index --`，变成了 2。成功删除了3

Index	0	1		2	3
Value	110	123		26	10

这是正确的，因为指向了下一个元素 26

再再注意，由于迭代器是一个内部类，它可以访问 `ArrayList` 的私有成员变量（字段）。

[5] 课后小问题

[问题 1] `remove()` is compulsory in Iterator implementation. (T or F)

翻译：在迭代器（Iterator）实现中，`remove()` 方法是必须的。（对或错）

答案：False。虽然 `Iterator` 接口定义了 `remove()` 方法，但在具体实现迭代器时，该方法不是强制必须实现的。如果迭代器不支持移除操作，可以在 `remove()` 方法中抛出 `UnsupportedOperationException` 异常。例如，在某些只读的迭代器实现中，就不需要实现实际的移除逻辑，只抛出异常即可。

[问题 2] How does ArrayList make use of the 'type parameter' in its implementation?

翻译：ArrayList 在其实现中是如何使用“类型参数”的？

在 Java 中，`ArrayList` 是一个泛型类，使用类型参数 `<E>` 来指定其存储元素的类型。

- 类型安全**：在编译时确存储存在 `ArrayList` 中的元素类型一致。例如，`ArrayList<String>` 只能存储 `String` 类型的对象，如果尝试添加其他类型对象，编译器会报错。
- 代码复用**：通过类型参数，`ArrayList` 可以适应不同类型的元素存储需求，而不需要为每种元素类型编写单独的类。例如，既可以创建 `ArrayList<Integer>` 存储整数，也可以创建 `ArrayList<Double>` 存储双精度浮点数。
- 自动装箱和拆箱**：当 `ArrayList` 存储基本数据类型的包装类时，会涉及自动装箱和拆箱。例如，`ArrayList<Integer>` 可以直接添加 `int` 类型数据，编译器会自动将 `int` 装箱为 `Integer`。

[问题 3] Which element will be removed by ArrayList.remove()?

翻译：ArrayList.remove() 会移除哪个元素？

`ArrayList` 有两个重载的 `remove()` 方法：

- `remove(int index)`：移除指定索引位置的元素。
- `remove(Object o)`：移除列表中首次出现的指定元素。例如

```
1 ArrayList<String> list = new ArrayList<>();
2 list.add("apple");
3 list.add("banana");
4 list.add("apple");
5 list.remove("apple");
```

会移除第一个出现的 "apple" 元素。如果元素不存在，该方法返回 `false`；如果成功移除元素，返回 `true`。

[问题 4] What does `ArrayList.next()` check before returning the next element in the list?

翻译: `ArrayList.next()` 在返回列表中的下一个元素之前会检查什么?

`ArrayList` 本身没有 `next()` 方法, `next()` 方法是 `Iterator` 接口中的方法, `ArrayList` 返回的迭代器实现了这个方法。在 `Iterator` 的 `next()` 方法返回下一个元素之前, 通常会检查以下内容:

是否还有下一个元素: 通过调用 `hasNext()` 方法来判断是否还有下一个元素。`hasNext()` 方法会维护一个内部的游标 (cursor), 判断游标是否已经到达列表末尾。如果 `hasNext()` 返回 `false`, 调用 `next()` 方法会抛出 `NoSuchElementException` 异常。例如:

[问题 5] How does `ArrayList.remove()` ensure only 1 element can be removed after each call to `next()`?

翻译: `ArrayList.remove()` 如何确保每次调用 `next()` 之后只能移除一个元素?

答案: 基于内部类迭代器实现, 维护一个变量 `canRemove` 来实现

[问题 6] What can happen if 2 or more Iterators running concurrently under the same `ArrayList`? Name 2 scenarios.

翻译: 如果在同一个 `ArrayList` 下同时运行两个或更多的迭代器, 会发生什么? 列举两种情况。

1. 数据不一致
2. 抛出 `ConcurrentModificationException`: Java 的 `ArrayList` 迭代器采用快速失败 (fail-fast) 机制。当一个迭代器检测到 `ArrayList` 在迭代过程中被结构修改 (如添加、移除元素, 除了通过迭代器自身的 `remove()` 方法), 会抛出 `ConcurrentModificationException` 异常。例如, 一个迭代器正在遍历, 另一个线程通过 `ArrayList` 的 `add()` 方法添加元素, 就可能触发这个异常。

Lecture 10

[6] 分析 Cost

我们如何分析一个程序、算法的Cost? 从 **时间** 和 **空间** 入手。

对于程序, 使用BenchMarking (基准测试)

对于算法, 使用复杂度分析

基准测试

测量内容

- **实际程序**：针对实际运行的程序进行考察。
- **在真实机器上**：在真实的物理计算机设备上开展测试，而非模拟环境。
- **针对特定输入**：使用特定的输入数据来运行程序。
- **测量耗时**
 - 通过 `System.currentTimeMillis()` 获取从系统时钟开始到当前的时间（以毫秒为单位，返回值类型为 `long`），以此来度量程序从开始运行到结束所花费的时间。
- **测量实际内存使用情况**：监测程序在运行过程中实际占用的内存量。

值得注意的问题：

- **选择合理的输入**：需要谨慎挑选测试数据集。应使用大规模数据集，并且**不要包含用户输入**。
- **其他用户 / 进程干扰**：其他用户运行的程序或系统中其他进程可能影响测试结果，要尽量减少这种影响。可通过多次运行程序并取平均值的方式来降低干扰。
- **使用哪台计算机**：需明确指定所使用计算机的详细信息，因为不同计算机的硬件配置会对测试结果产生显著影响。

复杂度分析

这是针对算法的。

将“步骤数”作为数据规模的函数来衡量。

- **最坏情况**（较容易分析）
- **平均情况**（较难分析）
- **最好情况**（容易分析，但用处不大）

构建步骤数的表达式：

- 代价 = $3.47n^2 - 67n + 53$ 步
- 代价 = $3n \log(n) - 5n + 6$ 步

简化为关于 n 的不同幂次 / 函数的项。

分析：渐近表示法

Asymptotic Notation

我们只关注当规模很大时的成本。

- **舍去低阶项**：（随着 n 增大，这些项变得无关紧要）
比如成本 = $3.47n^2 + \dots$ 步，成本 = $3n \log(n) + \dots$ 步。

- **忽略常数因子**：实际的常数因子取决于硬件。
所以舍去常数因子，得到 $\text{Cost} \propto n^2 + \dots$ 步，**成本** $\propto n \log(n) + \dots$ 步。
- **“渐近成本”，即“大O”成本**：描述成本如何随输入规模增长。
 - 成本为 $O(1)$ ：固定成本。
 - 成本为 $O(n)$ ：随 n 增长。

大O表示法

一种描述计算机算法效率的表示法：假设时钟频率为100兆赫兹， $N = 1024k = 2^{20}$ 。

- $O(1)$ - 常数时间，10纳秒。
- $O(\log N)$ - 对数时间，200纳秒。
- $O(N)$ - 线性时间，10.5毫秒。
- $O(N \log N)$ - $n \log n$ 时间，210毫秒。
- $O(N^2)$ - 二次时间，3.05小时。
- $O(N^3)$ - 三次时间，365年。
- $O(2^N)$ - 指数时间， 10^{10^5} 年。

如果数据/输入的规模为 n ，其增长情况如下：

$O(1)$ “常数级”：成本与 n 无关

$O(\log(n))$ “对数级”：规模乘以10，成本增加一点（增加 $\log(10)$ ）

$O(n)$ “线性级”：规模乘以10，成本变为10倍

$O(n \log(n))$ “ n 乘以 $\log(n)$ 级”：规模乘以10，成本变为略多于10倍

$O(n^2)$ “平方级”：规模乘以10，成本变为100倍

$O(n^3)$ “立方级”：规模乘以10，成本变为1000倍

...

$O(2^n)$ “指数级”：规模增加1，成本翻倍

=> 你肯定不想运行这种算法！

$O(n!)$ “阶乘级”：规模增加1，成本变为 n 倍

=> 你绝对不会想用这种算法！

函数的层次结构

我们可以定义一个函数层次结构，其中每个函数的数量级都比它的前一个函数大：

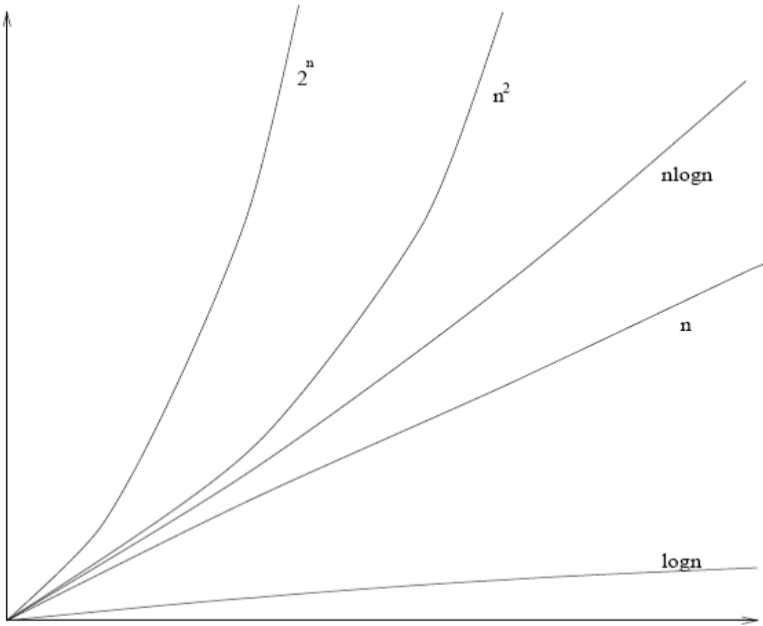
函数类型	函数示例
常数函数	1
对数函数	$\log n$

函数类型	函数示例
根号	$\sqrt{n}$ 比 $\log n$ 增长稍快
多项式函数	$n, n^2, n^3, \dots, n^k$ (还可细化如 $n \log n, n^2 \log n$ 等)
指数函数	2^n

这对于CPT104的学习很有帮助！是的，就是104... 这些层级不可逾越，简单来说，举个很极端的例子

$n^{999999999}$ 总会在某处 $< 2^n$

note: 阶乘归在了指数层级，但是阶乘还是大于指数



什么是步骤

Step

计数规则

- 最内层循环中的操作（执行次数最多）
- 每次循环都会执行的操作（不在“if”语句内）
- 涉及“数据值”的操作（而不是索引）
- 有代表性的操作（无需统计每一个操作）

```

1 public E remove(int index){
2     if (index < 0 || index >= count) throw new ...Exception();
3     E ans = data[index];
4     for (int i = index + 1; i < count; i++) // 在最内层循环中
5         data[i - 1] = data[i]; // 关键步骤
6     count--;
7     data[count] = null;
8     return ans;
9 }

```

每次 `for` 循环：

- 1 次比较操作： `i < count` （不计入）
- 1 次加法操作： `i++` （不计入）
- 1 次数据检索操作： `data[i]` （不计入）
- 1 次减法操作： `i - 1` （不计入）
- 1 次内存存储操作： `data[i - 1] = data[i]` （计入，因为这个是开销Cost最大的操作）

那么如何判断、统计开销最大的操作呢？

- 两个数字相加开销较小，但是求幂运算开销相对较大。
- 比较两个字符串可能开销较大。
- 从文件中读取一行内容开销可能**非常大**。
- 等待用户或其他程序的输入可能耗时极长.....

评估ArrayList的方法Method

操作	时间复杂度
get	$O(1)$
set	$O(1)$
remove	$O(n)$
add (at i)	$O(n)$ （最坏和平均情况）
add (at end)	$O(1)$ （平均情况）
add (at end)	$O(n)$ （最坏情况）
add (at end)	$O(1)$ （均摊 amortised 平均情况）

[7] Q&A

[题目1] $O(\log(n)) < O(\sqrt{n})$ (T or F)

翻译： $O(\log(n))$ 小于 $O(\sqrt{n})$ （对或错）

答案: T

解释: 随着 n 的增大, $\log(n)$ 的增长速度比 $\sqrt{n}$ 慢。例如, 当 n 足够大时, $\log(n)$ 的值会远小于 $\sqrt{n}$ 的值, 所以从渐进时间复杂度的角度, $O(\log(n))$ 表示的增长速度小于 $O(\sqrt{n})$, 该表述正确。

[题目2] $O(n^n) < O(n!)$ (T or F)

翻译: $O(n^n)$ 小于 $O(n!)$ (对或错)

答案: F

解释: $n! = 1 \times 2 \times \dots \times n$, 而 $n^n = n \times n \times \dots \times n$ 。当 n 增大时, n^n 的增长速度远快于 $n!$ 。可以通过极限等数学方法证明, 所以 $O(n^n)$ 大于 $O(n!)$, 该表述错误。

[题目3] $O(2^n) < O(n^n)$ (T or F)

翻译: $O(2^n)$ 小于 $O(n^n)$ (对或错)

答案: T

解释: 随着 n 的增大, 2^n 的增长速度远慢于 n^n 。因为 n^n 中底数和指数都随 n 增大, 而 2^n 底数固定为 2。所以 $O(2^n)$ 表示的增长速度小于 $O(n^n)$, 该表述正确。

[题目4] When analysing the cost of an algorithm, loop usually is the focus. (T or F)

翻译: 在分析算法成本时, 循环通常是重点。 (对或错)

答案: T

解释: 循环结构通常会重复执行某些操作, 其执行次数往往与输入规模相关, 会对算法的时间复杂度产生关键影响。许多算法的时间复杂度主要由循环的执行次数和循环内操作的复杂度决定, 所以在分析算法成本时, 循环通常是重点, 该表述正确。

[题目5] Which of the following operations is more expensive? Reading a line from a file; Reading a line from a user

翻译: 以下哪个操作成本更高? 从文件中读取一行; 从用户读取一行

答案: Reading a line from a user

解释: 从用户读取一行涉及到与用户的交互, 用户输入的速度不确定, 可能会有较长的等待时间, 而且可能需要处理用户输入的各种异常情况。而从文件中读取一行相对来说是在计算机存储系统内的操作, 速度较为稳定且通常较快。所以从用户读取一行成本更高。

[题目6] **Worst case cost analysis is usually more difficult than average cost analysis. (T or F)**

翻译：最坏情况成本分析通常比平均成本分析更困难。（对或错）

答案：F

解释：平均成本分析需要考虑所有可能输入的概率分布等因素，要准确评估每个输入情况及其发生概率比较困难。而最坏情况成本分析只需要找出导致算法性能最差的特定输入情况，相对而言，确定最坏情况可能更容易一些。所以通常平均成本分析比最坏情况成本分析更困难，该表述错误。